



Internal assessment

The internal assessment is a summative task for you to showcase your Computer Science skills and prowess. It is marked by your school teacher and moderated by the IB, prior to contributing to your final grade for the course. The *IBDP Computer science guide* provides these details on the internal assessment:

◆ **Solution:** the documentation and video submitted by the student for the internal assessment.

*The internal assessment requires the student to identify a problem of their own choice and develop a software **solution** using the computational thinking process.*

Adapted from the IBDP Computer science guide

For Standard Level students the internal assessment is worth 30 per cent of your final grade, and for Higher Level students it is worth 20 per cent of your final grade.

For all students, it is recommended that 35 hours of teaching time is allocated to work on the internal assessment. According to the *IBDP guide*, this time includes:

- time for the teacher to explain the requirements of the internal assessment
- class time for students to work on the internal assessment component and ask questions
- time for consultation between the teacher and each student
- time to review and monitor progress, and to check authenticity.

IBDP Computer science guide

■ The internal assessment can earn up to 30 marks, spread over five criteria

Criteria	Title	Marks	Recommended word count	Extras
A	Problem specification	4	300	
B	Planning	4	150	Diagrams
C	System overview	6	150	Diagrams
D	Development	12	1000	Video (max 5 minutes in length; format: MP4, AVI, WMV)
E	Evaluation	4	400	

Choice of problem

Making a wise choice of the problem you will tackle is critical, as the rest of the internal assessment flows from that. Don't rush into a decision without consulting closely with your Computer Science teacher. The *IBDP guide* states:

In identifying a problem, the student can select to apply to the problem any topic in computer science that interests them. It does not have to be directly related to the specified themes in the syllabus.

The problem chosen should require a software solution with sufficient complexity to be commensurate with the level of the DP Computer Science course. It should also require sufficient innovation for the student to demonstrate their organizational skills, algorithmic thinking and ability to code their algorithms.

IBDP Computer science guide

◆ **Product:** the completed software only (in the internal assessment).

The programming language and choice of technologies you use in preparing your **product** are not constrained to either Python or Java. You may use whichever language and technologies you deem to be the best fit for your product. Also, you may choose to develop a new computational solution: a standalone application; a computer or mobile game; an interactive website with a connection to a database; a mobile application; or even add functionality to an existing product. You should demonstrate problem-solving techniques combined with the use of data structures, logic conditions and data manipulation (via file processing or databases) using either a procedural or OOP approach.

Some examples of inappropriate products, from the *IBDP guide*, are:

- the development of a programming product using only copied code
- the development of a website (product) using a web-based template that determines its structure and layout
- the use of exemplar products or templates provided with software such as the Northwind database in MS Access
- a copied computer game without major modifications to the code that have been properly documented
- a product that does not meet the ethical requirements outlined in the 'Requirements and recommendations' section of the *IBDP guide*
- a computer / mobile application created using a builder / wizard / drag and drop tool without the need for code development.

IBDP Computer science guide

Additionally, examples of weaker products identified by IB moderators in the annual subject report for the previous version of the syllabus, which are still relevant to this version of the IA, include:

- Java programs that mainly focus on GUI and not on actual functionality
- Java programs that consist of one class only
- Java programs consisting of a Greenfoot template with only two methods overwritten
- Rudimentary versions of freely available games (like Sudoku)
- Websites that are template-based (Wordpress, Wix or Weebly) or that have minimal content.

IBDP Computer science subject report, May 2016, page 2

Top tips!

Choice of problem

Some general advice from the authors:

- Choose a problem that you are 80 per cent confident about being able to solve. This allows a good mix of confidently being able to deliver on what you set out to achieve, with a little bit of a stretch for you to challenge yourself with and learn something new through your IA.
 - A problem you are 100 per cent confident about from the start is probably too simple to allow for high grades.
 - A problem you are only 50 per cent confident about means you are trying to do more than is realistically feasible for your current skill set and available time. There are likely too many unknowns that could trip you up.
- Do not confuse a "large" project with a "complex" one. An example of this going wrong is creating a project that requires 10+ different GUI screens. One or two screens is enough to demonstrate you have the skill to create it; the rest is repetitious and wasted effort (from the perspective of the marking criteria).
- While on the subject of GUI screens, user interfaces generally do not require much algorithmic skill, so don't focus too much on the user interface itself beyond proper use of input validation techniques. Similarly, writing HTML or CSS does not, by itself, demonstrate algorithmic thinking.

Top tips!

- When selecting a problem, ask yourself where you will have the opportunity to showcase your understanding of both algorithmic thinking and data structures. When you consider the criterion D marking criteria, you will observe it is largely driven by algorithms, and by extension data structures. That said, your use of various algorithms and data structures must make sense in the context of your project – artificially forcing an algorithm into a project in a manner that appears contrived will not satisfy the criteria.
- Higher achieving products will typically have some technical complexity (being demonstrated through algorithms or data structures) beyond that learned through the course syllabus. Remember: the course guide allocates 35 hours of class time to this IA and, while it is an assessment, all assessments are also learning experiences. The IB does not intend for you to go through 35 hours of class time without learning anything. Seek out new skills to learn and implement in your IA.
- To contrast the previous statement, however, do not attempt a scenario that requires technical abilities beyond your capabilities. You should not introduce unnecessary complexity if a simpler solution is at hand. Simple code is elegant code.
- Do not neglect the importance of the documentation and video. Under the previous syllabus, it was very achievable for a product of moderate complexity to achieve a 7 overall on the basis of excellent, thorough, detailed documentation. Likewise, too many technically outstanding and brilliant products have ended up with low grades because students spent all their time programming and neglected the written work. As the *IBDP Computer science guide* states: “Whichever problem and form of solution a student chooses, it is essential that the student explicitly demonstrates and documents their algorithmic thinking skills.”
- You are not constrained to the content of the syllabus in selecting a problem. In fact, the *Guide* specifically states: “the student can select to apply to the problem any topic in computer science that interests them. It does not have to be directly related to the specified themes in the syllabus.”

Project ideas and inspiration

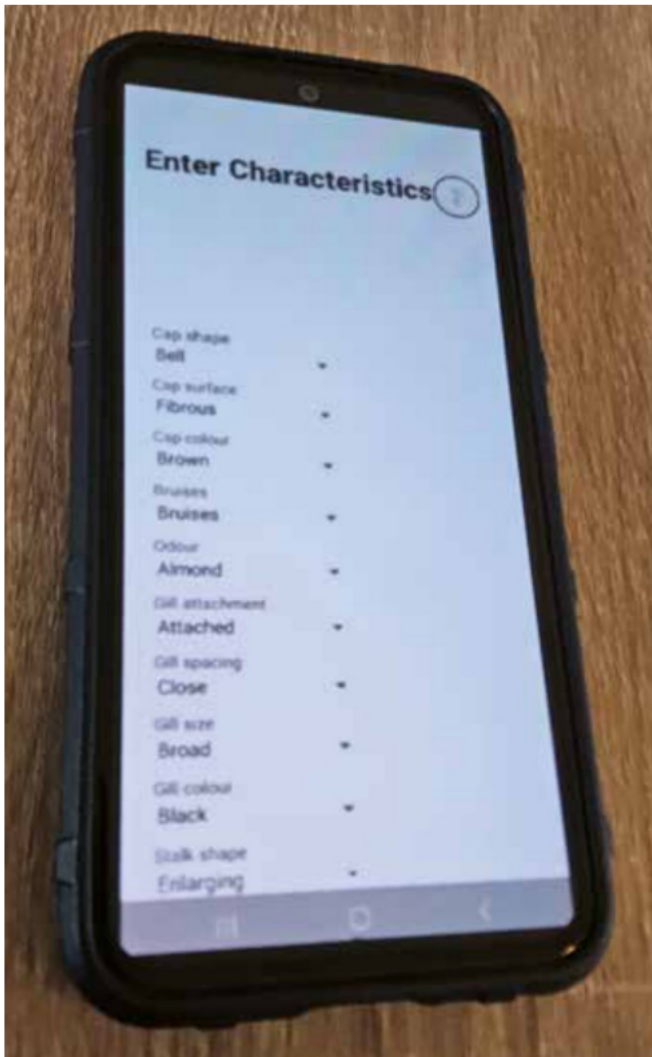
The following overviews are projects that the authors’ students have had success with. All of these projects received a post-moderation grade of 7 (27+ points). While aspects of the projects would need to be adapted for the updated syllabus, they are still instructive and valuable for illustrating the variety of projects with which students can find success.

The authors wish to express their appreciation to these former students for their willingness to have their projects shared in this book. Several of the example diagrams that appear in later sections of this chapter are also from these former students’ IA projects.

■ Mushroom: Friend or foe?

by H Ng, used with permission

This project aims to develop a mobile application to assist users in identifying mushrooms. The app will utilize a machine learning model trained on a North American data set to determine whether a mushroom is edible, based on its features. The app will also provide educational resources to help users learn more about mushrooms. The goal is to create a user-friendly and informative tool for mushroom foragers.



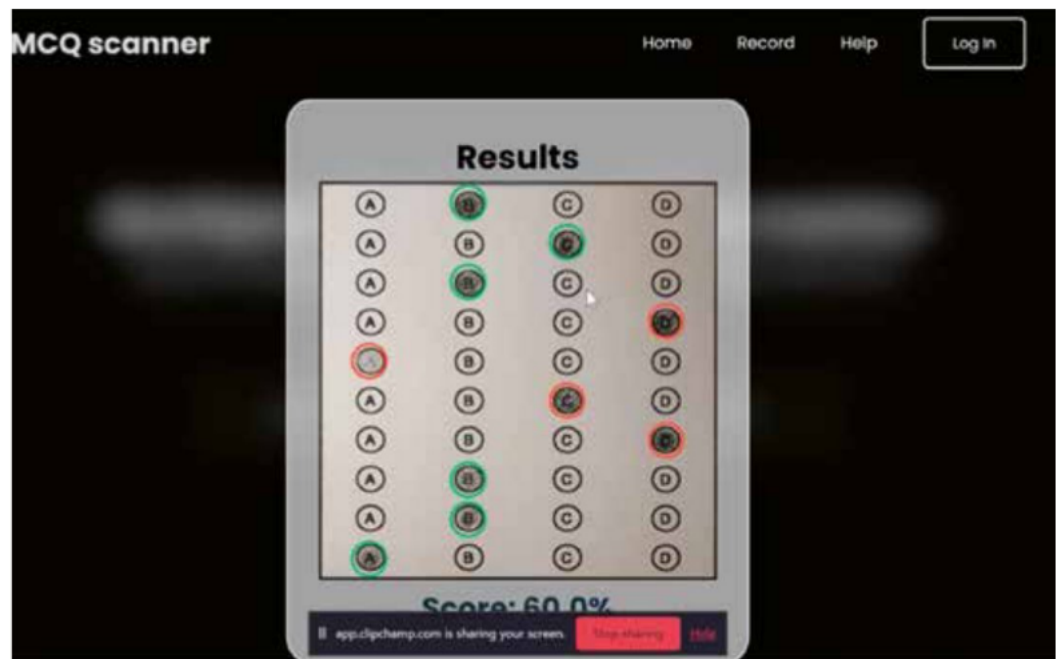
■ Mushroom: Friend or foe?

Key technologies and techniques

- **Machine learning:** Classification models (e.g. logistic regression, decision trees) for mushroom-edibility prediction
- **Python:** Programming language for machine learning model development
- **Java:** Programming language for Android app development
- **Graphical user interface (GUI):** For user interaction and data input
- **Server-client architecture:** To separate the machine learning model and the mobile app
- **Data structures:** JSON to represent mushroom features and model predictions
- **Error handling:** To handle unexpected inputs or errors gracefully
- **Web development:** Flask framework for building the web API
- **Mobile app development:** Techniques for creating Android apps, including layout design, resource management and testing

■ Automated grading tool

by a former student, used with permission



■ Automated grading tool

This project aims to develop a web-based application to automate the grading of multiple-choice assessments. The application will use Optical Mark Recognition (OMR) to identify student answers from scanned images and compare them to a model answer sheet. Key features include automatic scoring, test saving and a user-friendly interface. The goal is to save teachers time and improve the efficiency of the grading process.

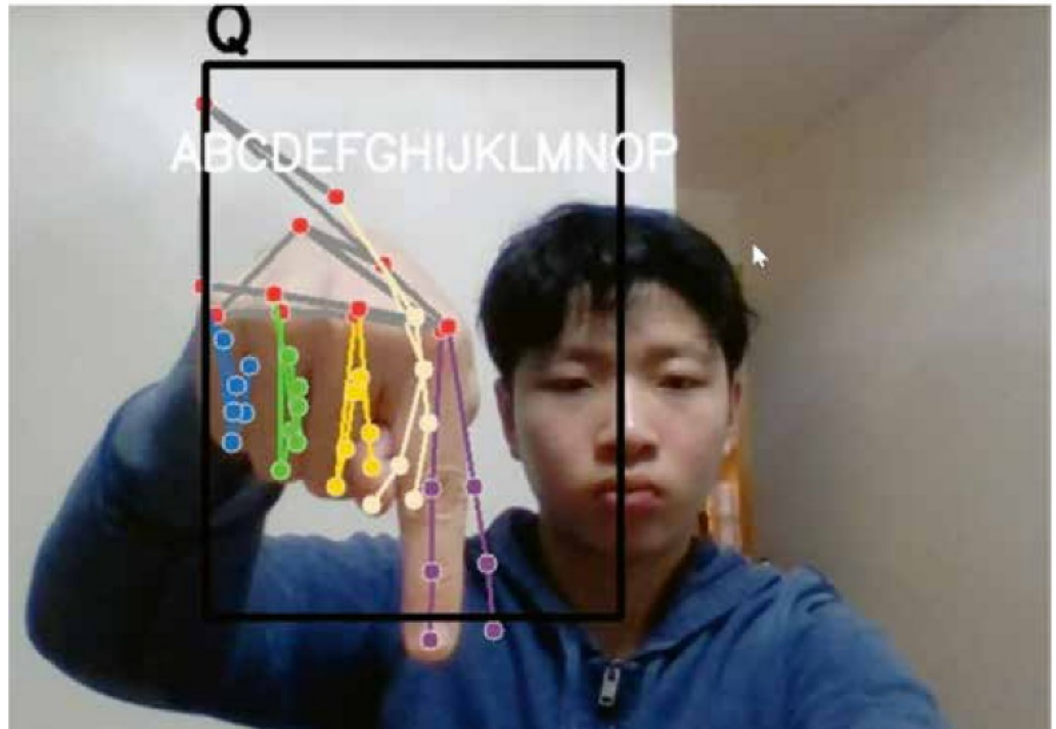
Key technologies and techniques

- **Web development:** Flask framework for building the web application
- **Optical Mark Recognition (OMR):** Techniques for identifying marked areas on scanned images
- **Computer vision:** OpenCV library for image processing and analysis
- **Python:** Programming language for overall implementation
- **SQLite database:** For storing user data and test results
- **User interface design:** Creating a user-friendly interface for uploading images, viewing results and managing tests
- **Error handling:** Implementing mechanisms to handle potential errors during image processing or data storage
- **Data structures:** Using appropriate data structures to represent test data, scores and user information
- **Image processing:** Techniques for preprocessing images, such as thresholding and noise reduction
- **Server-side scripting:** Handling user interactions, processing image data and updating the database

■ ASL (American Sign Language) interpreter

by Jared Xin, used with permission

Live Streaming - e to append, backspace to delete and f to add space



■ ASL (American Sign Language) interpreter

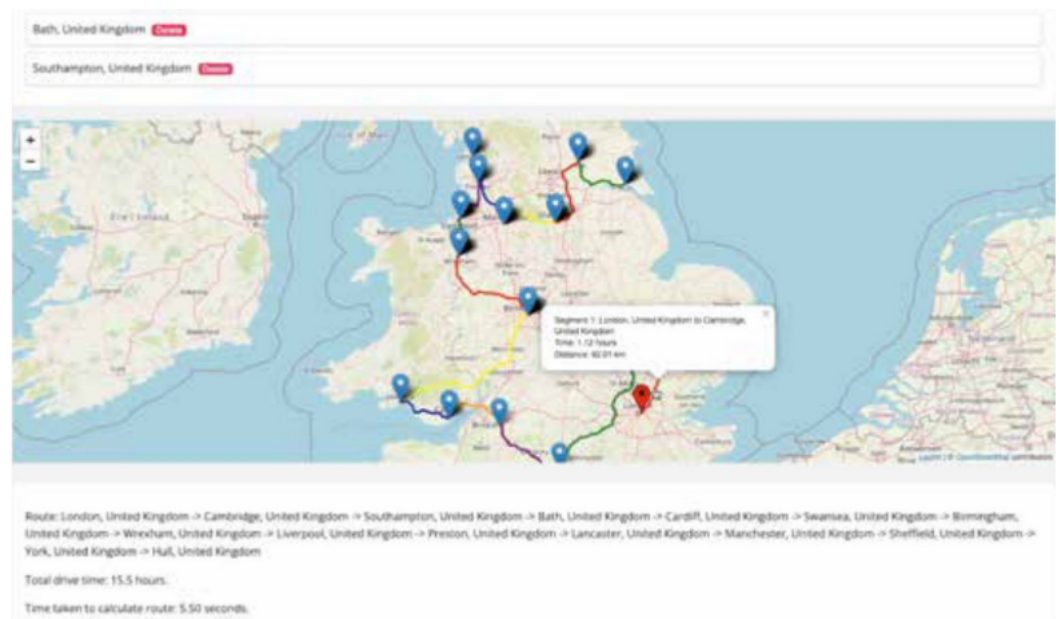
This project aims to develop a web-based application to facilitate communication between hearing and deaf individuals by providing real-time translation of American Sign Language (ASL) into text. The application will use machine learning and computer vision techniques to analyse hand gestures and predict the corresponding ASL letters. Key features include a user-friendly interface, live video-streaming and accurate translation. The goal is to improve accessibility and communication for the deaf community.

Key technologies and techniques

- **Web development:** Flask framework for building the web application
- **Machine learning:** Algorithms for predicting ASL letters from hand gestures
- **Computer vision:** OpenCV library for image processing and hand detection
- **Python:** Programming language for overall implementation
- **Webcam integration:** Techniques for accessing and using the user's webcam
- **User interface design:** Creating a visually appealing and user-friendly interface
- **Real-time processing:** Ensuring efficient processing of video frames for real-time translation
- **Data structures:** Representing hand shapes and predicted letters
- **Error handling:** Implementing mechanisms to handle potential errors during image processing or model prediction
- **Model training:** Training the machine learning model on a data set of ASL signs
- **Accuracy evaluation:** Evaluating the model's performance and making improvements

■ Vacation route planner

by Edward Zhang, used with permission



■ Vacation route planner

This project aims to develop a web-based application to assist users in planning multi-destination vacations. The application will allow users to input locations, generate optimized routes and view detailed information about each location. Key features include efficient route planning, location suggestions and mobile compatibility. The goal is to simplify the vacation planning process and provide users with personalized recommendations.

Key technologies and techniques

- **Web development:** Flask framework for building the web application
- **Mapping:** Leaflet.js library for interactive maps
- **Routing algorithms:** Travelling salesperson algorithm to calculate the most efficient routes between locations
- **Location data:** APIs to access and process location data using OpenStreetMap API
- **User interface design:** Creating a user-friendly interface for inputting locations, viewing routes and interacting with the map
- **Error handling:** Implementing mechanisms to handle potential errors during data processing or API requests
- **Data structures:** Representing locations, routes and other relevant data
- **Mobile optimization:** Ensuring the application is compatible with mobile devices
- **Deployment:** Using PythonAnywhere for deployment and hosting
- **Server-side scripting:** Handling user interactions, processing location data and generating routes

■ IBDP subject recommender

by Sofia Cornu, used with permission

The screenshot shows a web application titled "Subject Select" with a graduation cap icon. The main heading is "Please input your most recent semester grades". There are eight rows, each for a subject and its grade. The subjects are: Math Extended, Physics, Chemistry, Digital Design, Product Design, English, Humanities, and French Phase 5. The grades are: 7, 6, 5, 4, 3, 2, 1, and 7. A "Back to Main" button is in the top right, and a "Continue" button is in the bottom right. A status bar at the bottom indicates "Screen Video Recorder is sharing your screen" with "Stop sharing" and "Hide" buttons.

Subject	Grade
Subject 1: Math Extended	7
Subject 2: Physics	6
Subject 3: Chemistry	5
Subject 4: Digital Design	4
Subject 5: Product Design	3
Subject 6: English	2
Subject 7: Humanities	1
Subject 8: French Phase 5	7

■ IBDP subject recommender

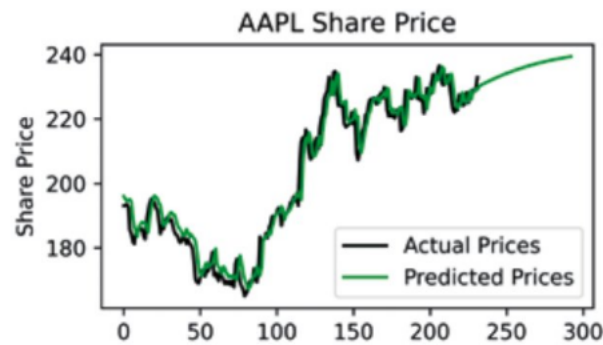
This project aims to develop a desktop application that will allow students to input information such as their subject interests, career aspirations and recent grades to receive a recommendation of which subjects they should choose for their IB diploma. The solution will take into account IB subject requirements and the school's timetable scheduling.

Key technologies and techniques

- Java programming language
- JavaFX GUI
- MySQL database to hold the school's timetable
- Excel file containing IB subjects that are recommended for different career paths, converted to JSON for easier use
- **Unique algorithm:** Developed to give students suggestions for their IBDP choices based on the data input

■ Stock-trading predictor

by a former student, used with permission



■ Stock-trading predictor

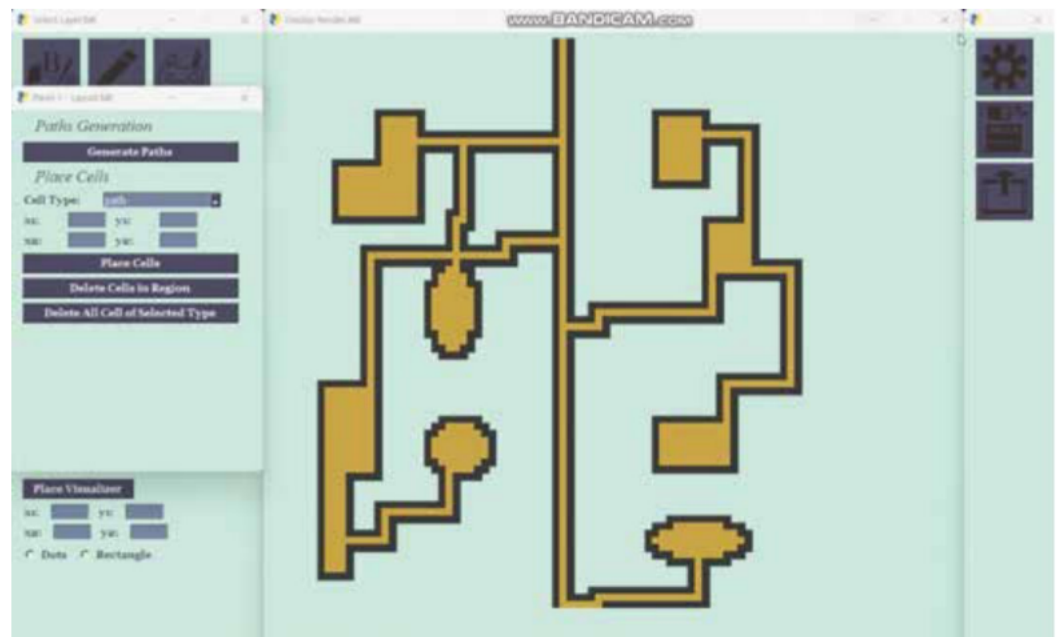
This project aims to develop an AI-powered tool to assist in making more informed stock-trading decisions. The tool will use a long short-term memory neural network to predict stock prices based on historical data. The AI models will be trained individually for each stock and saved to the user's computer. The program will feature a simple GUI with minimal UI elements and will provide error handling for invalid inputs. The tool will also allow users to retrain existing AI models.

Key technologies and techniques

- **AI:** Long short-term memory neural networks for stock-price prediction
- **Python:** Programming language for implementation
- **yFinance API:** For accessing stock data
- **Tkinter:** For creating the GUI
- **matplotlib:** For displaying graphs
- **Machine learning libraries, e.g. TensorFlow or PyTorch:** For neural network training and implementation

■ D&D battle map generator

by Tom Chan, used with permission

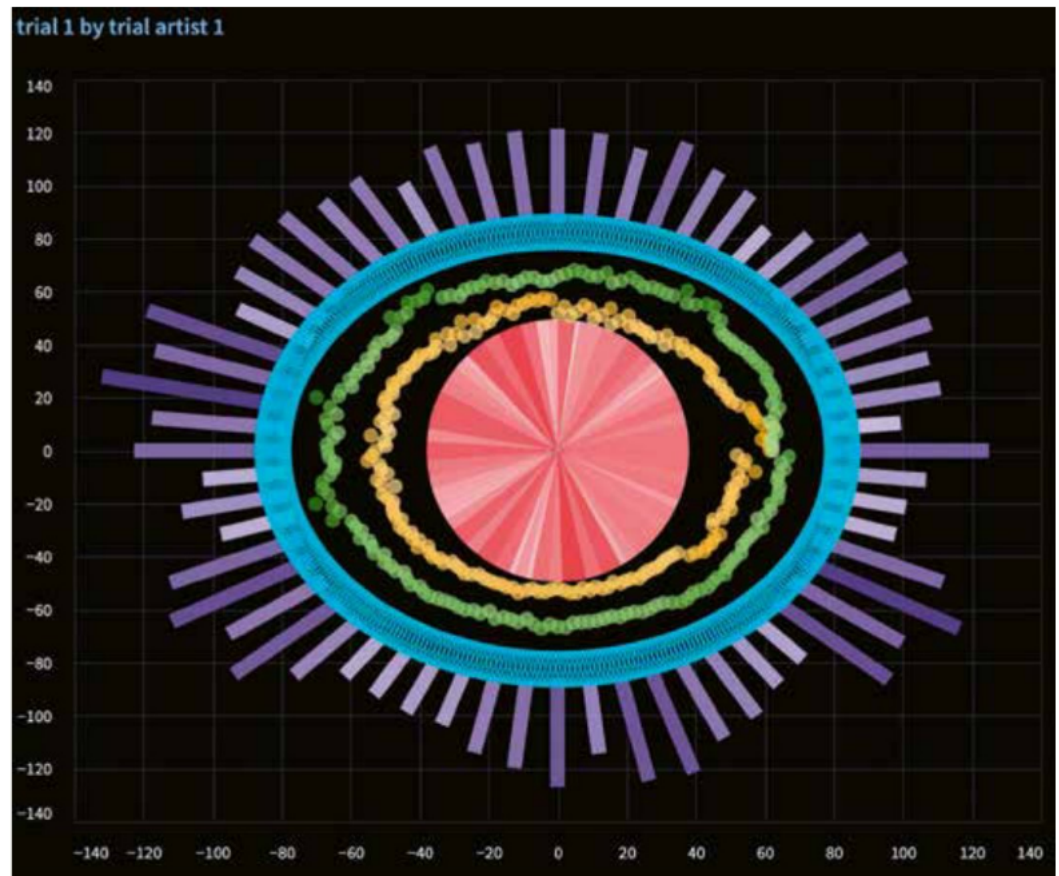


■ D&D battle map generator

This project aims to create a desktop application that generates custom battle maps for Dungeons & Dragons. The application will feature a user-friendly interface, allowing users to customize maps easily with various textures and elements. The goal is to provide a quick and efficient solution for creating unique battle maps. Key features include procedural map generation, texture import, and easy saving and exporting.

Key technologies and techniques

- **Python:** Programming language for implementation
- **PySimpleGUI:** Front-end framework for creating the user interface
- **PIL (Pillow):** For image processing and rendering
- **Procedural generation algorithms:** To generate structures, paths and hazards
- **Algorithms:** Breadth first search, A* path finding, recursive back-tracking
- **Data structures:** OOP to represent the map, rooms and paths
- **Texture mapping:** To apply textures to the generated map
- **File I/O:** For saving and loading maps
- **Image export:** For exporting maps as png, jpeg or pdf



■ Example Visuca output

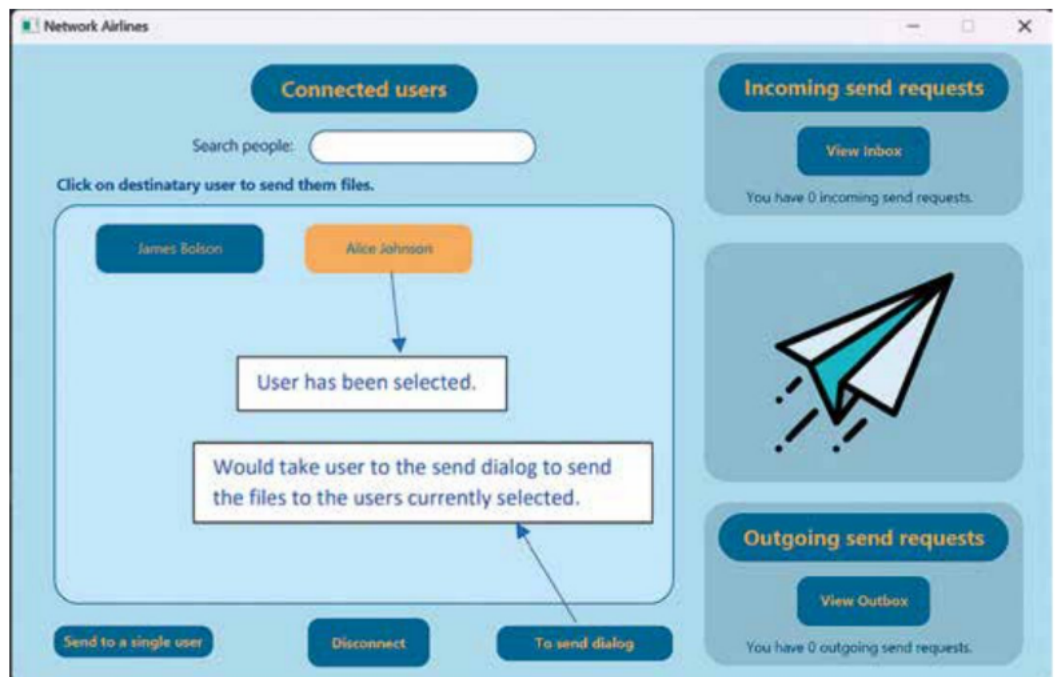
This project aims to create a desktop application that will generate a poster based on the musical elements of a song. The application will request the upload of a csv file with the musical element details and will have instructions for the user to convert their song of choice into a csv file. The application will then take into consideration the musical elements to create different posters that vary based on shape, pattern and colour intensity. The poster designs will change based on the client's song choice, so the deconstructed musical elements are represented as a 'visual display'.

Key technologies and techniques

- **Python:** Programming language for implementation
- **Streamlit:** Front-end framework for creating a user interface
- **Librosa:** Python library for extracting music data from MP3s
- **Vega-Altair:** Python library to create visual output
- **Algorithms:** Unique algorithm to display the music data visually
- **Data structures:** OOP to hold music data for individual tracks, lists and Panda data frames
- **File I/O:** Text files and Pickle library to import and export data

■ Network airlines (file transfer system)

by Eladio Hosseinpour, used with permission



■ Network airlines (file transfer system)

This project aims to create a file transfer system that downloads files from one computer to another on the same network, through the ethernet – which connects devices on the local network – rather than the internet, so the data can travel directly between computers on the same network rather than having to be uploaded and downloaded, which is inefficient.

Key technologies and techniques

- **Java:** Programming language for implementation
- **JavaFX:** Front-end framework for creating a user interface
- **Multi-threading:** To boost the efficiency of the application
- **Asymmetric encryption:** Ensuring transmission security
- **Recursion:** Used for searching directories and sorting data
- **Algorithms:** Network scanning
- **Data structures:** OOP to hold file data and user information

Submission requirements

There are three files that will be submitted at the conclusion of your internal assessment: the documentation, the video and the appendices. Briefly, the role of each is as follows:

■ Documentation

- Submitted as a single PDF file
- Should contain five separate sections, one for each criterion
- Total word count must not exceed 2000 words (not including code excerpts, comments or diagrams)
- The overall word count should be clearly written on the cover page.

■ Video

- Maximum length of five minutes
- Submitted in a commonly used file format such as MP4, AVI or WMV
- Demonstrates the full functionality of the product
- Demonstrates examples of the testing strategy used in the development of the product.

■ Appendices

- Submitted as a single PDF file
- Must include the full source code and other resources developed that are referred to in the documentation
- While appendices are not used as evidence for the awarding of marks, and examiners are not required to read them, solutions that do not include an appendix with full source code cannot be awarded full marks for techniques demonstrated in criterion D.

Criterion A: Problem specification

The *Guide* provides the following information on criterion A:

The problem specification is the starting point of the solution and must be used as a basis for the development of the product.

The student should have the necessary technical skills, access to appropriate hardware and software and the availability of relevant data to address the problem.

- The success criteria identified in the problem specification (assessed by criterion A) will be used in the planning (assessed by criterion B), in the development (assessed by criterion D) and in the evaluation (assessed by criterion E).
- The recommended word count for this criterion is 300 words.
- Criterion A assesses the problem specification (4 marks)

Marks	Description
0	The response does not reach a standard described by the descriptors below.
1–2	Outlines a problem scenario. States limited success criteria. Outlines the nature of the solution in a computational context.
3–4	Describes the problem scenario in terms of its measurable solution requirements. States appropriate success criteria. Explains the choice of computational context for the solution.

Clarifications:

- **Problem scenario:** The problem scenario is a clear description of the problem, including its measurable solution requirements. The description should relate directly to the problem, whether this be in the world around us, in other fields of knowledge or a current issue in computing.
- **Success criteria:** These are measurable outcomes derived from the solution requirements that indicate the successful development of the product.
- **Computation contexts:** The computational context is the specific area of computing that is selected to be used in the solution.

IBDP Computer science guide

Top tips!

Criterion A

- 1 Use the SMART technique to be as specific as possible with your success criteria (Specific, Measurable, Achievable, Relevant, Time-bound).
- 2 Use technical language in your success criteria (in the correct context). Vague, generic success criteria are unlikely to score marks.
- 3 Ensure you are not just recreating a clone of an existing app, such as a game. There must be an original idea, with an element of innovation, to your project. Be sure to articulate what sets your project apart from all the others. The IB provides some questions that may help:
 - ☐ What is the current situation?
 - ☐ Who is affected by the problem?
 - ☐ What causes the problem?
 - ☐ What are the objectives or general requirements for the computational solution?
 - ☐ Are there any constraints associated with the problem?

- ☐ Are there existing computational solutions and, if so, why are they not effective? What are their shortcomings?
- ☐ Why is it valuable or significant to construct a computational solution for the problem?
- ☐ Are there any time or resource limitations or specific conditions?

IBDP Computer science teacher support material

- 4 Examples of good success criteria include:
 - ☐ The solution will be able to input, edit and delete customer records from the customer database.
 - ☐ The solution will give appropriate warning messages to the user in case of extreme or invalid input.
 - ☐ The solution will be menu-driven with eight menu choices (including an indication of the choices).
- 5 Examples of inadequate success criteria include:
 - ☐ The code will compile.
 - ☐ The code will run without crashing.
 - ☐ The code will use loops.
 - ☐ The program will be aesthetically pleasing.

Common mistakes

Criterion A

- 1 **Criteria for success that do not actually make reference to the core functions of the proposed solution** (for example, creating a task manager without any mention of the core functions associated with being a task manager): This will always imply there are key criteria absent from this list. If you are designing a game, include some aspects of the game-play in your CfS – it will make your testing far easier.
- 2 **Make sure your success criteria are testable and not subjective:** A success criterion such as “The UI should look good” is not suitable, as it is vague and subjective and people may disagree on the outcome.
- 3 **Your success criteria have a key role in later assessment criteria:** You will design tests for them, and evaluate your project based on them, so ensure your success criteria meaningfully summarize the core functional requirements for your project.

■ Checklist for criterion A

■ Problem scenario:

- ☐ The purpose, or idea, inspiring your problem solution has been clearly identified and described.
- ☐ Explicitly relates to one of: the world around us; another field of knowledge; or a current issue in computing.
- ☐ Measurable solution requirement(s). What is the essence or core functionality for your project? Summarize the purpose of your project here in a way that can be objectively measured, and then expand on it in the success criteria.

■ Computational context:

- ☐ Clearly identify your choice(s) of computational context. This may include:
 - Language environment – which programming language will you use and why? What are any potential flaws or weaknesses with this choice and how are they not a factor, or how have they been mitigated?
 - Software environment – specific operating systems, libraries, frameworks to be used. Justify your choice.
 - Hardware environment – specific components or computing infrastructure to be used/required. Justify your choice.
 - Data environment – format, structure, size and source of the data you will be using or generating; any database engine you may use.
 - Implementation environment – system configuration, network conditions, security constraints, resource availability and compatibility requirements.
- ☐ Explain and justify why you have chosen these computational context(s).
- ☐ If you have not yet finalized your choices, you can identify the options you are evaluating and what your guiding concern may be in making a choice. Be sure to identify this as a research task in your criterion B planning.
- ☐ You may consider using a table such as the one below to help structure your computational context.

■ Table to help structure your computational context

Context	Chosen	Alternatives considered	Justification
Main language	Java	Python	Because ...
Hardware environment			
Libraries / frameworks			
OS			
Data environment			

■ Success criteria:

- ☐ Related to the problem scenario previously described
- ☐ Clearly measurable, with precise language
- ☐ Testable
- ☐ Achievable and feasible within the constraints of your skills and time available
- ☐ The sum of the success criteria will indicate successful development of the product
- ☐ 8 to 10 in number
- ☐ Should be application-feature focused rather than what programming techniques will be used.

■ 300 words.

■ Examples for criterion A

These are based on two of the projects outlined in the 'Project ideas and inspiration' section, reworked for the updated assessment criteria.

Vacation route planner

Based on the project by Edward Zhang

Problem scenario:

As a frequent traveller, I've struggled with planning multi-destination vacations, particularly across different countries. Existing tools like Google Maps and Tripadvisor lack the functionality to effectively sequence travel routes and offer limited customization, presenting a significant challenge in the travel and tourism sector. This problem highlights a gap in **current computing** solutions concerning data handling and real-time optimization for personalized travel planning. My project shall consist of a web-based application that allows users to select different cities and landmarks they wish to visit, creating a "tour" as a list of destinations. The application shall use route optimization and personalization features (such as travel method) to generate recommended itineraries.

Computational contexts:

The proposed solution is a web-based travel-planning application using **Flask** for back end operations due to its simplicity and effectiveness in managing routes and endpoint setups, and the ease with which it allows use of Python for back end functionality while using web technologies for a modern, stylish front end. The front end will utilize **HTML** for structure, **CSS** for styling and **JavaScript** for functionality, while incorporating the **Leaflet.js** library for advanced interactive mapping capabilities. This stack was chosen to ensure broad compatibility across devices, essential for real-time travel use. The combination of HTML/CSS/JS is industry standard for UI and there is plenty of documentation available to assist in creating the front end with these tools. The application will be hosted on **PythonAnywhere** due to its tooling being specifically designed for hosting Python projects with ease, in contrast with setting up a VPS with AWS or similar. This setup provides the flexibility needed for a personalized, dynamic travel-planning tool that meets the needs of myself and my friends.

Word count: 257

Success criteria:

The project will be considered successful if it meets the following criteria:

- 1 Generates a near optimal route (with respect to estimated travel time) connecting user-selected locations, starting and ending at user-specified points, using the travelling salesperson algorithm.
- 2 Displays the calculated route within 10 seconds for up to 15 locations.
- 3 Accurately recognizes and suggests locations based on partial or non-English inputs.
- 4 Provides interactive map functionalities with detailed markers and route segments.
- 5 Integrates with APIs to ensure accurate and current routing and location data.
- 6 Displays clear error messages for unreachable locations or invalid inputs.
- 7 Alerts users to input locations before attempting route planning.
- 8 Manages user inputs effectively, preventing entry of more locations than supported and prompting corrections.
- 9 User interface designed for mobile phone browsers, specifically the current version of iPhone Safari.

IBDP subject recommender

Based on the project by Sofia Cornu

Problem scenario:

As students entering the International Baccalaureate (IB) Diploma Programme, my friends and I are challenged by selecting subjects that align with our interests, career goals and IB requirements, while avoiding timetable conflicts. This relates to the **world around me** as it directly affects the planning of my future education through the ability to combine personal preferences with logistical constraints. The current manual approach, reliant on individual counselling, is inefficient and inadequate. Highlighting problems in educational logistics and computing, such as data integration and user personalization, there's a clear need for a digital solution that systematically addresses these challenges, enhancing educational planning through advanced computational techniques.

As such, I will create a solution that allows aspiring IB students to enter previous report grades, and personal preference data, from which the solution will generate a proposed subject enrolment that is personalized for the student. The solution will also ensure the proposed subject enrolment is compatible with the school's IB timetable offerings and blocking structure.

Computational contexts:

The application will be developed using **Java** due to its robust **static typing** system, which reduces bugs and enhances performance. Java's compatibility with **JavaFX** makes it ideal for creating a user-friendly **desktop environment** that operates seamlessly across **multiple operating systems**. Data management will be handled using a **MySQL server**, facilitating real-time updates and secure storage of timetable and student information. This approach leverages Java's capabilities for complex data structures and secure, scalable application development, addressing the specific needs of educational administration and compliance with educational standards.

Word count: 248

Success criteria:

The application must meet the following criteria to be considered successful:

- 1 **Personalization:** Users can input personal interests, career goals and academic performance. The application will recommend subjects based on these inputs.
- 2 **Scheduling integration:** It accesses the school's timetable data to ensure recommended subjects do not clash.
- 3 **Compliance with IB requirements:** Ensures the proposed subject combination is valid for the IB Diploma, including the correct level distribution (three Higher Level and three Standard Level subjects).
- 4 **User interface:** Features a graphical user interface (GUI) that allows for easy input of personal information using text boxes, check boxes and drop-down menus.
- 5 **Flexibility:** Includes a feature for selecting optional group 6 subjects and considers 'Environmental and Social Sciences' as applicable to multiple groups.
- 6 **Security:** Only authorized personnel (e.g. the IB coordinator) can alter timetable settings, secured via password protection.
- 7 **Cross-platform compatibility:** Functions on both Windows 10 and MacOS, aligning with the technology available to the students.

Criterion B: Planning

The *Guide* provides the following information on criterion B:

The planning of the product must be consistent with the problem specification in criterion A.

- This criterion assesses how the problem scenario has been decomposed into component parts.
- The plan should address the requirements of the solution, in terms of the success criteria, and include a proposed chronology for the steps involved in planning, designing, developing, testing and evaluating the solution.
- A plan can be presented in different forms, but diagrams such as Gantt and Agile charts can effectively support the planning process.
- The plan may include the allocation of time toward conducting research into code libraries, frameworks or other tools that may be suitable for the project.

The recommended word count for this criterion is 150 words.

■ Criterion B assesses the planning (4 marks)

Marks	Description
0	The response does not reach a standard described by the descriptors below.
1–2	Constructs a partial decomposition of the problem scenario. Constructs a plan that addresses some of the success criteria of the solution.
3–4	Constructs a reasonable decomposition of the problem scenario. Constructs a plan that addresses the success criteria of the solution.

Common mistakes

Criterion B

- 1 **Producing diagrams that are not technically accurate:** For any diagram you produce, make sure it complies with the standard for that type of diagram.
- 2 **Plans that are later contradicted by the solution submitted,** for example if the plan states you will use Microsoft SQL but you then end up using SQLite.
- 3 **Missing areas of the design cycle,** for example not showing any evidence of testing being carried out on a component.

Clarification of planning:

- **Decomposition** is the breaking down of the problem scenario identified in criterion A into smaller, more manageable sub-problems or components. The decomposition can be effectively constructed using diagrams.
- A **reasonable decomposition** breaks the problem down into essential components that support the construction of a plan.

IBDP Computer science guide

Top tips!

Criterion B

- 1 Show you have carried out the design cycle for each sub-problem / component (planning, designing, developing, testing and evaluating).
- 2 There are multiple tools to facilitate decomposition, for example:
 - ☐ The general solution can be documented by a use case diagram together with flowcharts, dataflow diagrams or equivalent.
 - ☐ For procedural coding, it may be good to use a flowcharting tool to break down a complex task into modules.
 - ☐ For object-oriented coding, it is important to identify and describe the objects in the problem scenario.
 - ☐ A database project would require the identification and description of the data and processing required.
- 3 The plan should take into account the order of tasks, the timeline for their completion and the resources needed to complete the individual tasks. This is also a time when any research that is required can be planned for.

IBDP teacher support material, page 76

■ Checklist for criterion B

Decomposition

- Each of the success criteria from criterion A has been identified within the decomposition process.
- You may use a table or bullet point format. Alternatively, a structure chart with short comments underneath can be a useful means to articulate the different sub-parts of your problem (see example).
- Optionally you may also include a high-level overview of some key logic (such as a flowchart) or your data structures (such as an OOP relationship diagram). If included, these are “big picture” only as detailed design diagrams occur in criterion C.

Planning

- The plan should address all five stages:
 - ☐ Planning
 - May include relevant research required for completion of the product (e.g. code libraries, frameworks, algorithms)
 - ☐ Designing
 - ☐ Developing
 - ☐ Testing
 - ☐ Evaluating
- Each of the sub-parts identified in the decomposition should be shown to have time allocated within your plan for designing, developing, testing and evaluating.
- Time limits and resources are considered to assign realistic timeframes and needs.
- You may, optionally, use planning diagrams such as a Gantt chart to present your planning (see the example).
- 150 words.

The IB offers this useful reminder:

Note that the plan is not a process journal. While an Agile approach is encouraged, the planning stage of the documentation should not be written after the computational solution is completed.

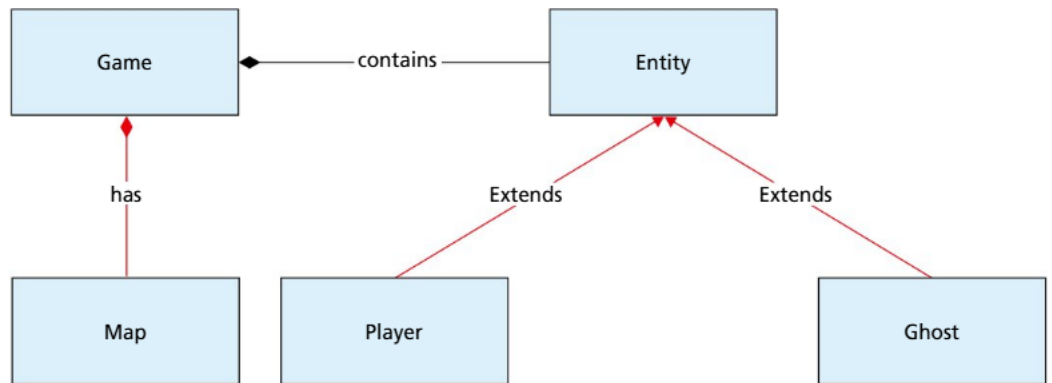
■ Example for criterion B

The following example has been constructed for a hypothetical, personalizable Pacman clone game. Remember that, at this stage, you are only decomposing and planning your problem. These diagrams will not necessarily reflect the final version of your solution.

Project: Pacman clone

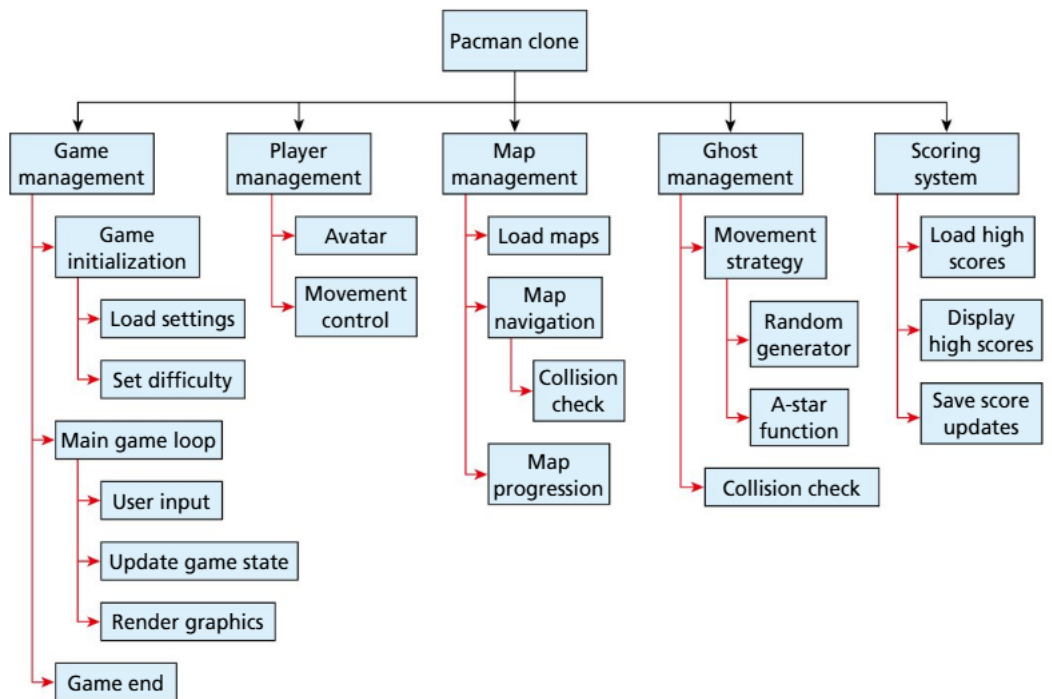
Decomposition

For my Pacman clone project, I have decomposed the problem into five main objects of Game, Map, Entity, Player and Ghost, as modelled in the UML relationship diagram that follows. Additionally I have decomposed the core functionality of the project according to the structure chart that follows (brief descriptions of each category follow the structure chart).



■ Class relationships UML

Structure chart identifying the key components of the decomposed project:

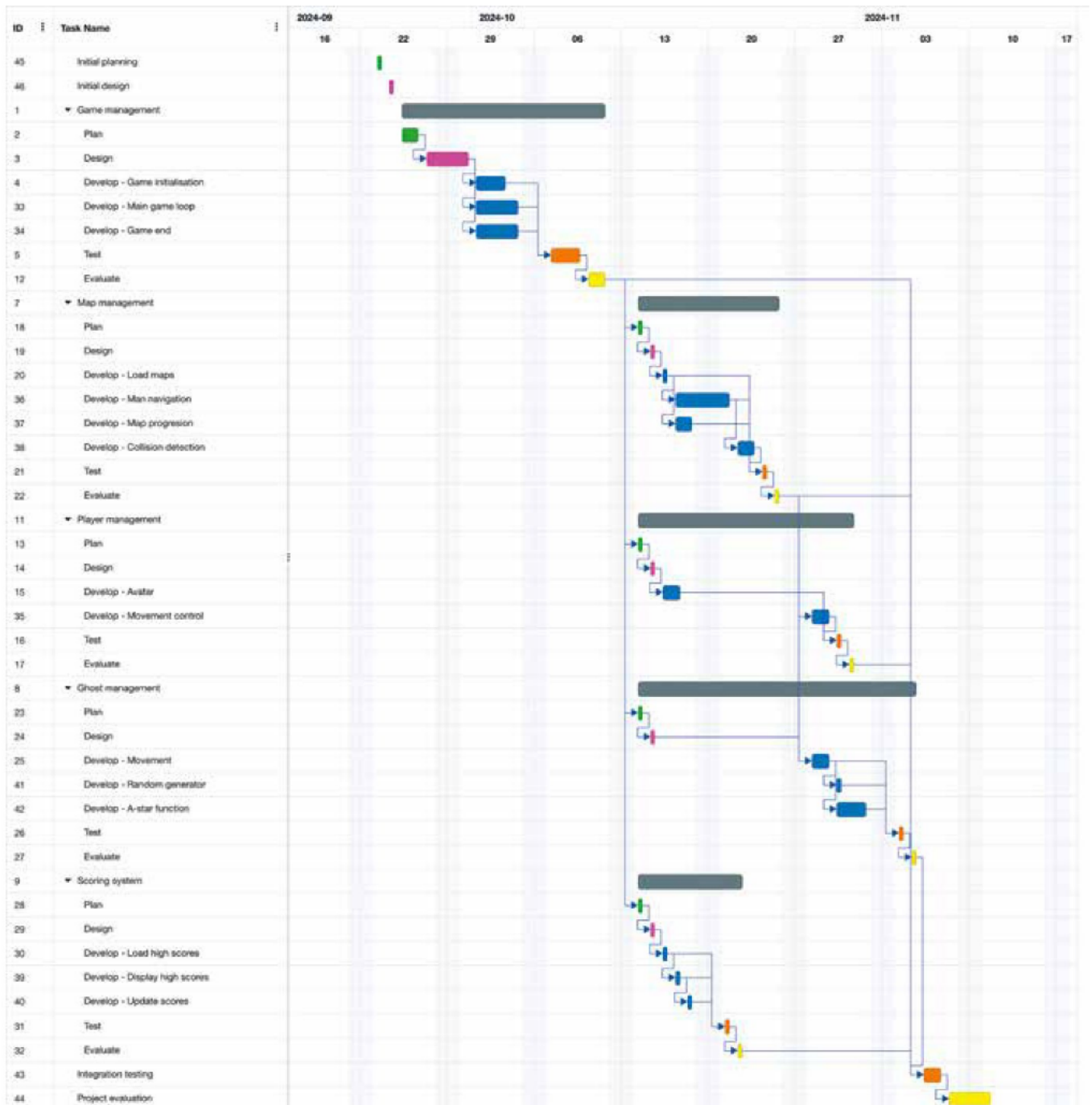


■ Structure chart

Further descriptions of the key components in the structure chart:

- **Game management:** This module handles the core loop and initialization, making sure the game starts and ends appropriately.
- **Player management:** Manages everything related to the player character, from avatar customization to handling movements based on keyboard inputs.
- **Map management:** Responsible for loading different maps and managing elements within them, including transitioning between different difficulty levels.
- **Ghost management:** Controls the ghosts' behaviour, utilizing an A-star algorithm for pathfinding and deciding their movement strategy based on the current difficulty level.
- **Scoring system:** Manages the scoring, including calculating scores based on game events (e.g. eating fruits) and handling high scores' storage and retrieval.

Gantt chart:



■ Gantt chart, illustrating the planning, designing, developing, testing and evaluation of each of the key components identified in the decomposition process and shown in the structure

Criterion C: System overview

The *Guide* provides the following information on criterion C:

The system overview of the product must be consistent with the problem specification in criterion A, and the planning in criterion B.

- The system overview should include a system model with the key components; their relationships; the rules governing their interaction; and the algorithms required by these components and the user interface.
- The system overview should have the clarity to enable a third party to re-create the product.
- The system model should provide the information for a viable testing strategy.

The recommended word count for this criterion is 150 words.

■ Criterion C assesses the system overview (6 marks)

Marks	Description
0	The response does not reach a standard described by the descriptors below.
1–2	Outlines a limited system model. Identifies algorithms for the components of the system model. Identifies a testing strategy for at least one success criterion.
3–4	Constructs a system model that is not complete . Constructs algorithms for the components of the model that lead to partial functionality of the product. Outlines a testing strategy that aligns with at least three success criteria.
5–6	Constructs a complete system model. Constructs algorithms for the components of the system model that enable the product to perform . Describes a testing strategy that aligns with the success criteria.

System overview clarifications:

- A **system model** consists of diagrams that include the components of the system and how they are connected. The system model will include the design of the User Interface. A **complete system model** does not include the algorithms for each of the components.
- **Algorithms** can be presented in different forms, including natural language, flowcharts or pseudocode, and should address the individual components of the system model.
- The **testing strategy** refers to a systematic approach for evaluating whether the computational solution works as intended. The testing strategy should ensure that code functions correctly and handles unexpected or incorrect inputs. This can be represented effectively in a table with proposed test data and expected outcomes.

IBDP Computer science guide

● Top tips!

Criterion C

- 1 Testing strategies should be explicitly identified for each of your criteria for success. Functionally test and verify that your software functions according to requirements. Test that the range of inputs and outputs aligns with expected behaviour.
- 2 Never trust the user! As discussed in A2.4.3, user inputs should always be validated before being used for any calculations or algorithms. Common validation methods for user inputs include:
 - ❑ **Presence check:** Has a value been input?
 - ❑ **Length check:** Is the input an acceptable length (minimum or maximum length)?
 - ❑ **Type check:** Does the input correspond to the correct data types: integer, float, absence of illegal characters?
 - ❑ **Format check:** Have any complex formatting rules been correctly applied (such as dates, times or currency)?
- 3 Validate your inputs by supplying expected and unexpected input values to test how your software responds. Normal data, boundary data and erroneous data inputs should all be tested. Remember that the guiding principle for this assessment criteria in the *Guide* is:

the system overview should have the clarity to enable a third party to re-create the product

Does your response provide that level of clarity?
- 4 Recommended diagrams:
 - ❑ UI – mandatory
 - ❑ ERD (normalized) – if you have a database
 - ❑ Data dictionary – if you have a database or persistent file storage
 - ❑ UML diagram – if you are using OOP anywhere
 - ❑ Component design diagram – if you have any physical hardware components
 - ❑ Rules of engagement, attack surface mapping – if you have networking involved
 - ❑ Flowcharting – for procedural code.

● Common mistakes

Criterion C

- 1 **A lack of design thinking being applied to your solution:** If you have genuinely decomposed your problem scenario, there really shouldn't be blocks of code hundreds of lines long (in criterion C, this could present as pseudocode that is overly long). This could be interpreted as a lack of effort to adequately think through and decompose your problem into abstractions such as classes or functions. The presence of variable names such as `person1`, `person2`, `person3` may be interpreted as something that could have been better designed with an array.
- 2 **Avoid any technical errors in your diagrams** (for example flowchart diagrams that have two arrows coming out of a non-decision element).
- 3 **Tests not actually relevant to achieving the proposed solution** (likely linked to poorly devised success criteria): Things like "starts up within 10 seconds" or "does not crash", instead of genuinely testing functionality, such as "successfully adds record to the XXX table of the database when a message is received".

◆ **Functional testing:** testing concerned with the behaviour of the application – specifically, whether it meets the requirements specified. This type of testing evaluates the software by providing inputs and examining the outputs, without considering how internal systems work. You are testing each of the success criteria on the whole application from the user's point of view.

◆ **Structural testing:** testing concerned with the internal workings of the application – based on the code structure and internal pathways. This type of testing requires an understanding of the codebase and is used to ensure that all aspects of the code are properly tested. You are testing that all conditional branches execute correctly, and all error-handling code triggers when needed and responds appropriately.

■ Checklist for criterion C

■ System model – holistic overview:

A system overview presents key components, their relationships and the rules governing their interaction. Some appropriate tools to facilitate this are:

- ☐ **Procedural coding:** Flowcharting to break down modules or a bulleted list with indentations.
- ☐ **OOP:** UML diagrams that show dependencies between classes.
- ☐ **Database project:** Table designs with normalization and a description of data dictionaries.

■ System model – for each component:

Students are expected to show algorithms for key components only. There is no need to design all components of the system.

- ☐ Diagrams for each component, including the design of the user interface.
- ☐ Algorithms are not required for all components, but should be provided for those that lead to the functionality of the product. These algorithms may be presented using either natural language, flowcharts or pseudocode.

■ Testing strategies:

Your document should contain two test tables:

- 1 **Functional testing table:** This table should evaluate the application against all success criteria, ensuring that the functionality meets user expectations and requirements.
- 2 **Structural testing table (white box testing):** This table should rigorously test major algorithms, using valid, extreme and invalid data to ensure the application performs correctly under all possible conditions.

■ Functional testing:

Address all elements of the success criteria.

■ Structural testing:

- ☐ Verify the major algorithms function correctly.
- ☐ Verify the solution handles unexpected or incorrect inputs and that you have tested the acceptable boundaries (valid, extreme and invalid data).

Examples for criterion C

The following tables provide a suggested format for documenting your testing strategy:

■ Functional testing

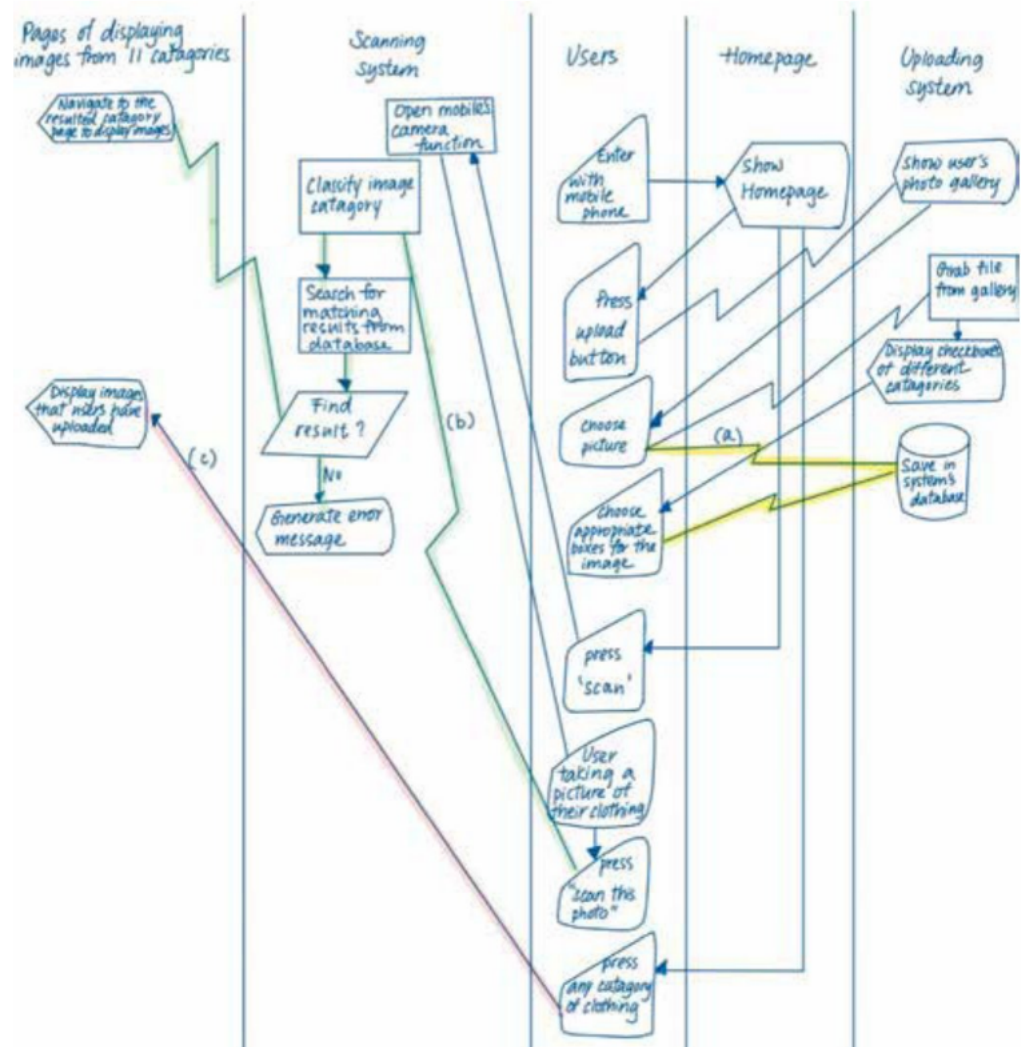
Success criteria	Test number	Test	Expected outcome
1	1	Upload files of different types and sizes	All files upload correctly and are resized to a resolution of 300x300
2	2		
3	3	Load the game with a single ghost and observe its movement	Ghost should move to Pacman using a sensible path
4	4		
5	5	Run the game, and use the WSAD keys to observe Pacman movement	Movements should be: W = up S = down A = left D = right

■ Structural testing

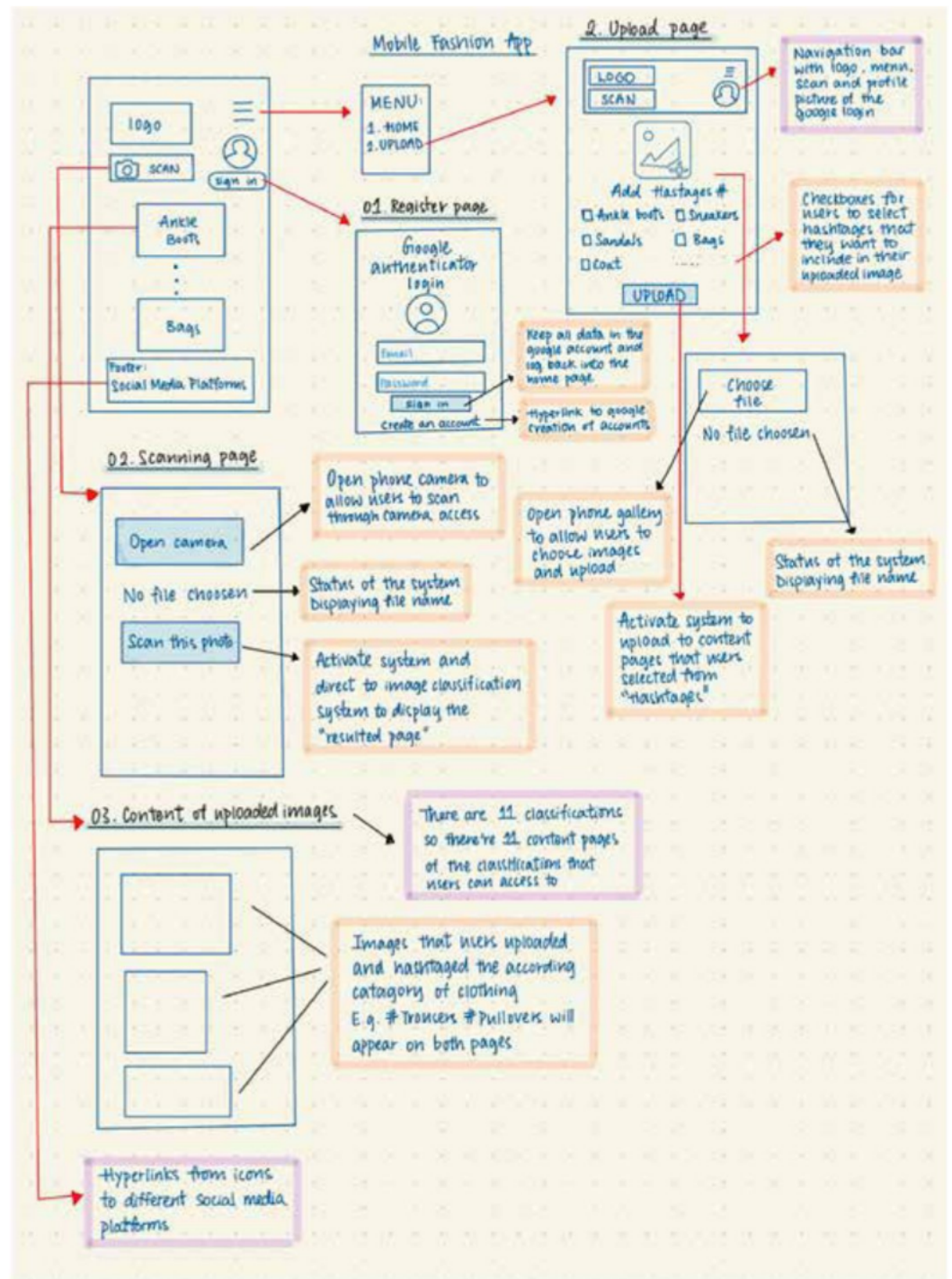
Algorithm / structure	Test	Test type	Test data	Expected outcome
Uploading avatars	Upload files of different types and sizes	Valid	A mix of PNG and JPG files of resolutions: 100×100 500×500 1000×1000 2000×2000	Image file is correctly uploaded. Post-upload, the code resizes to a 300×300 avatar, and saves to internal memory for use within the game.
		Invalid	Non-image files uploaded, e.g. docx, xlsx, pdf	Error message displayed to the user with instructions about permitted image sizes and types.
		Extreme	File over 5MB in size, and / or dimensions over 2000×2000	Error message displayed to the user with instructions about permitted image sizes and types.
A-star algorithm	Unit testing on the <code>A_Star()</code> function	Valid	A range of source and target coordinates that are valid locations in the maps	Unit testing should pass these tests when expecting the correct next closest coordinate to be returned.
		Invalid	A range of source and target coordinates that are out of bounds / located in a wall / otherwise not valid locations in the maps	Unit testing should fail these checks.
		Extreme	Test on: map with no pathway from source to target; also test with source and target being at the same location	Unit testing should fail these checks.
User input controls	Use unittest.mock to simulate a range of keyboard inputs and combinations	Valid	A mix of W, A, S, D keyboard inputs	Game should correctly process the input.
		Invalid	Inputs of other keys	Game should ignore the input.
		Extreme	Rapid repeated key presses or simultaneous key presses	Game should ignore the input.

■ Example diagrams for criterion C

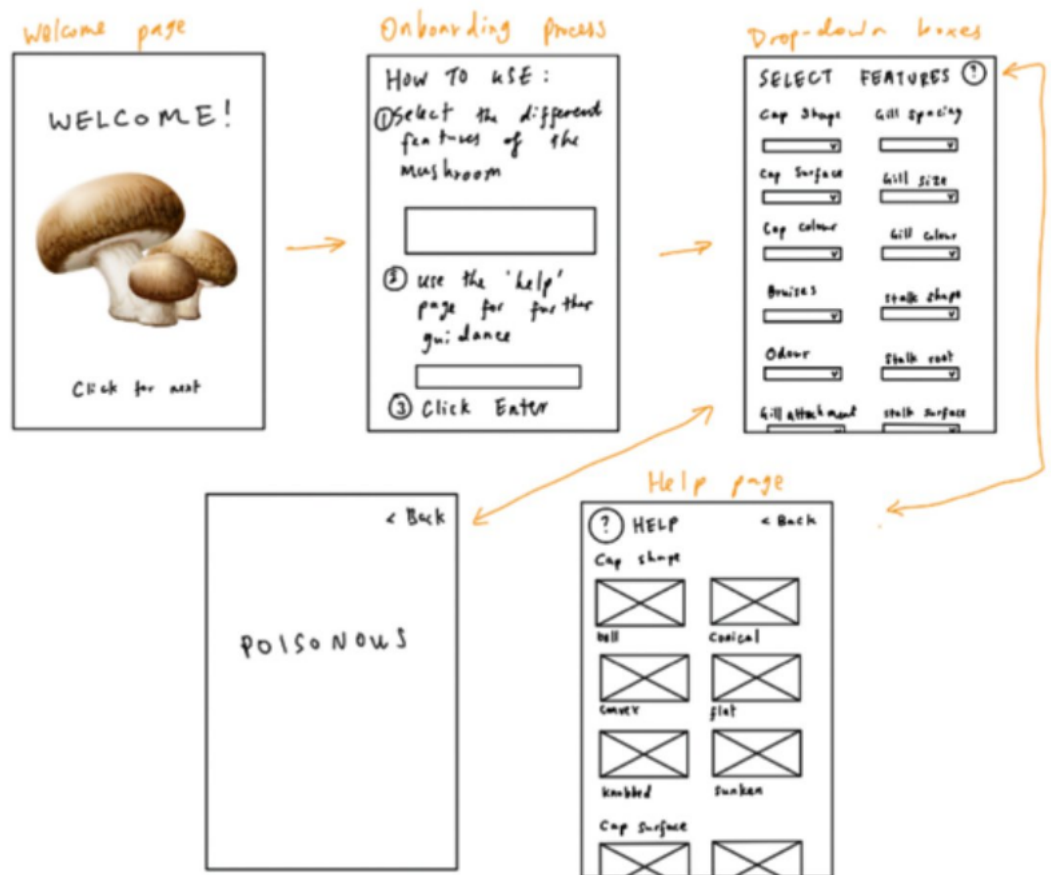
The following are exemplar diagrams of the type that may be commonly included for this criterion (depending on your project choice).



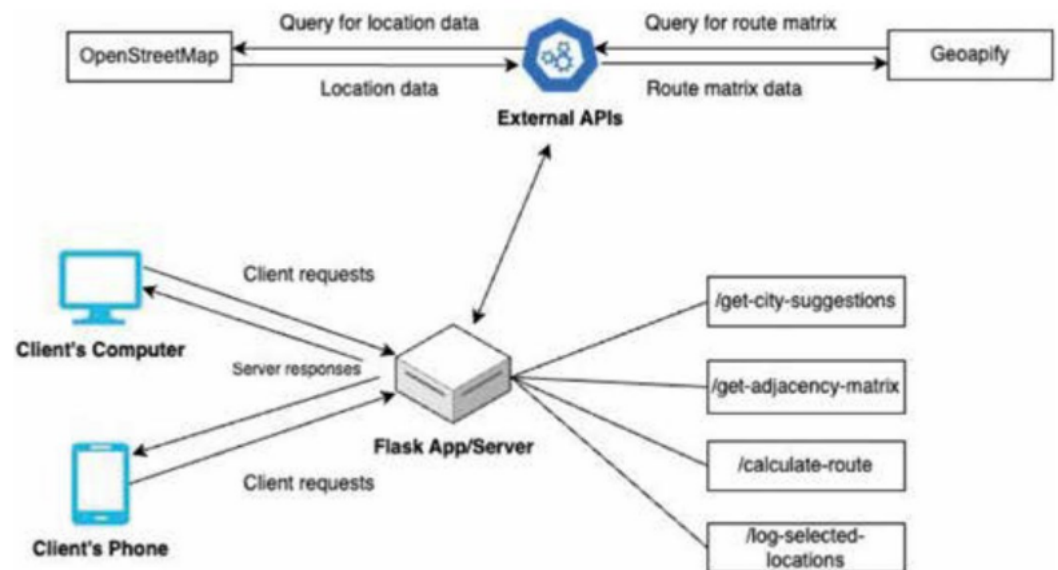
■ System flow diagram (credit: Hailey Annabelle Loh)



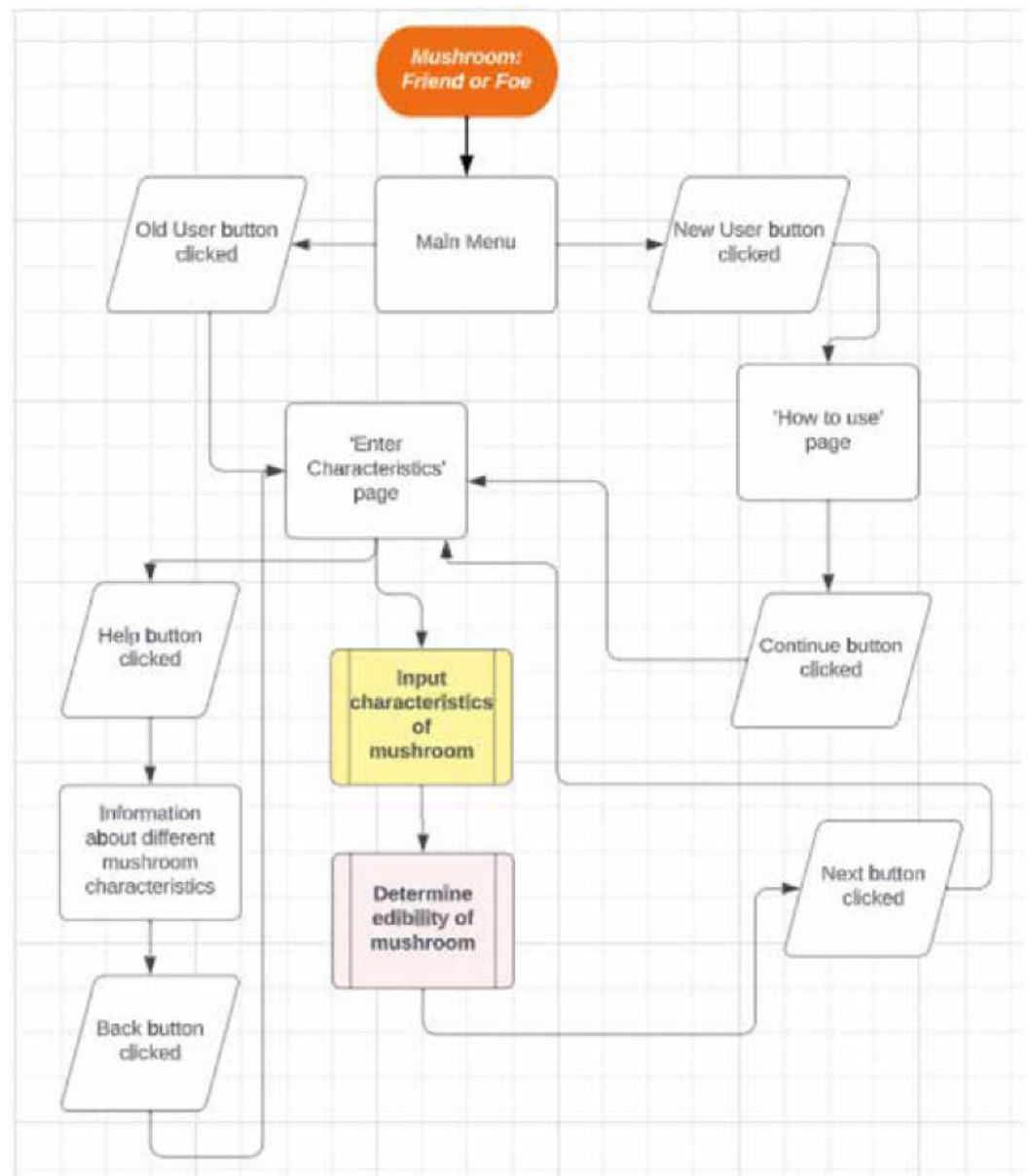
■ User interface mock-up 1 (credit: Hailey Annabelle Loh)



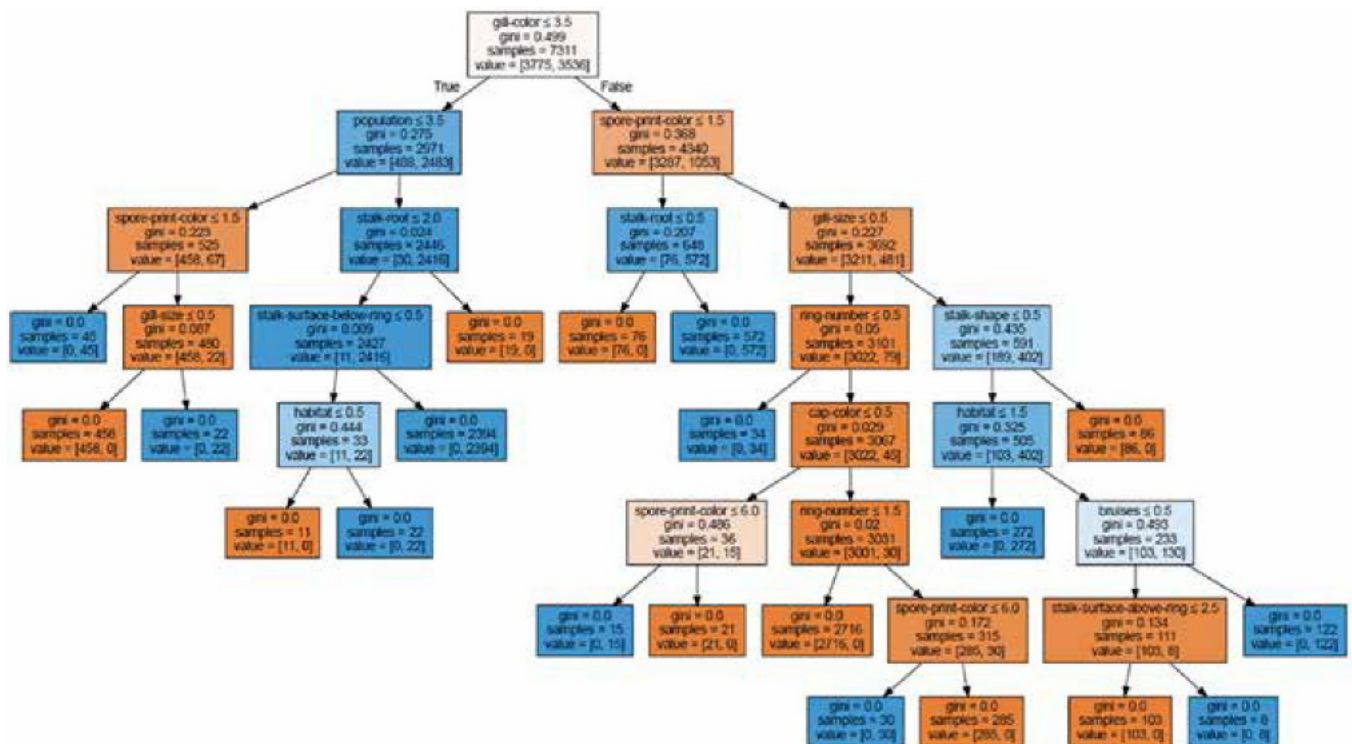
■ User interface mock-up 2 (credit: H Ng)



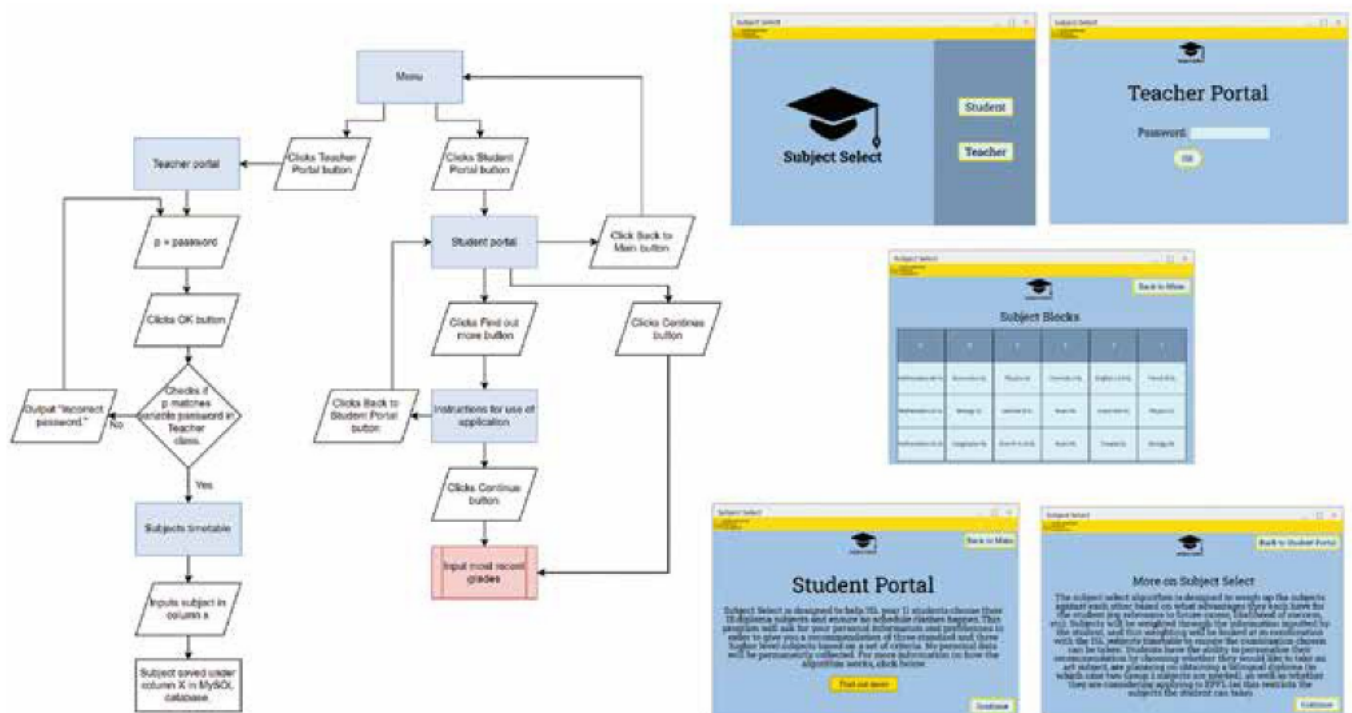
■ Network architecture diagram (credit: Edward Zhang)



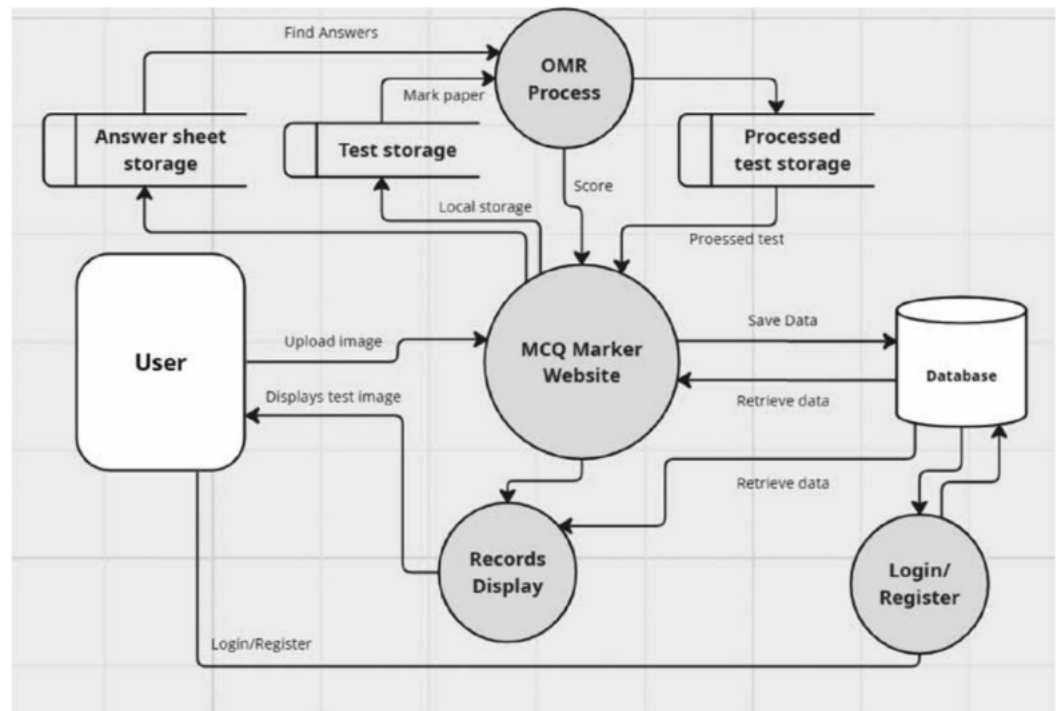
■ High-level overview flowchart (credit: H Ng)



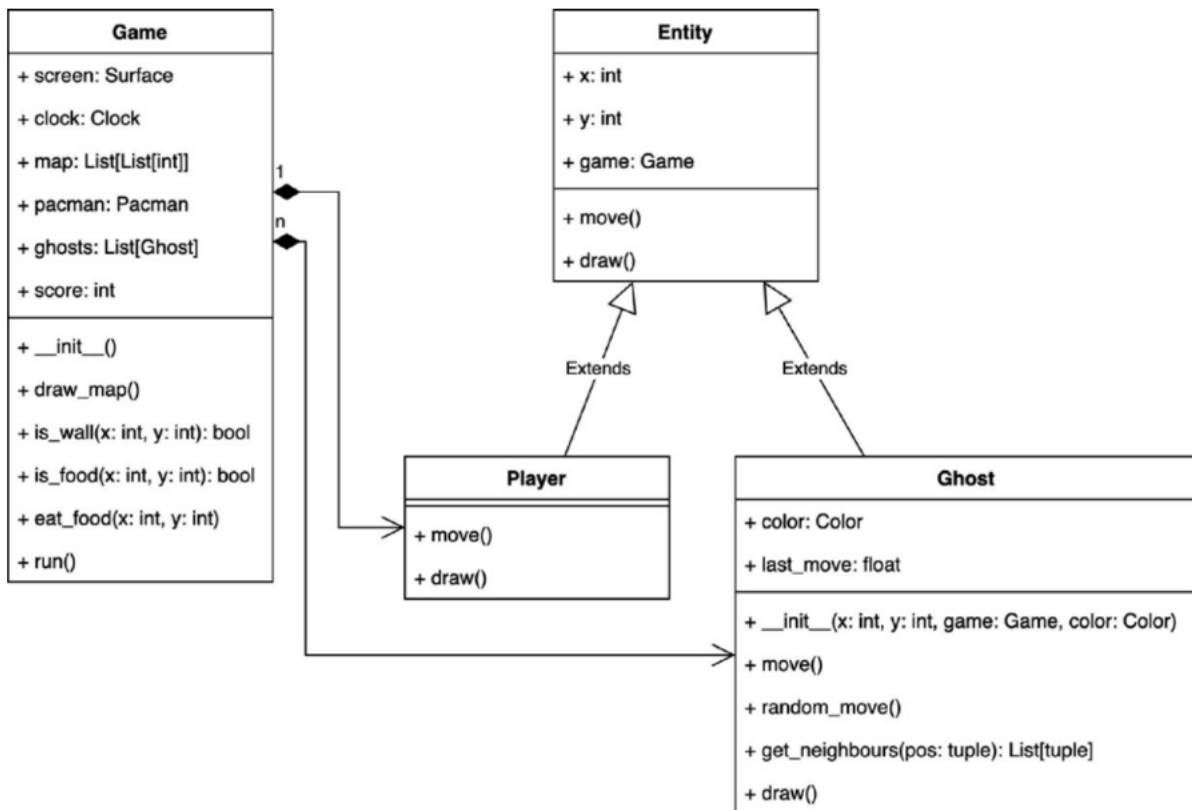
■ Decision tree ML model (credit: H Ng)



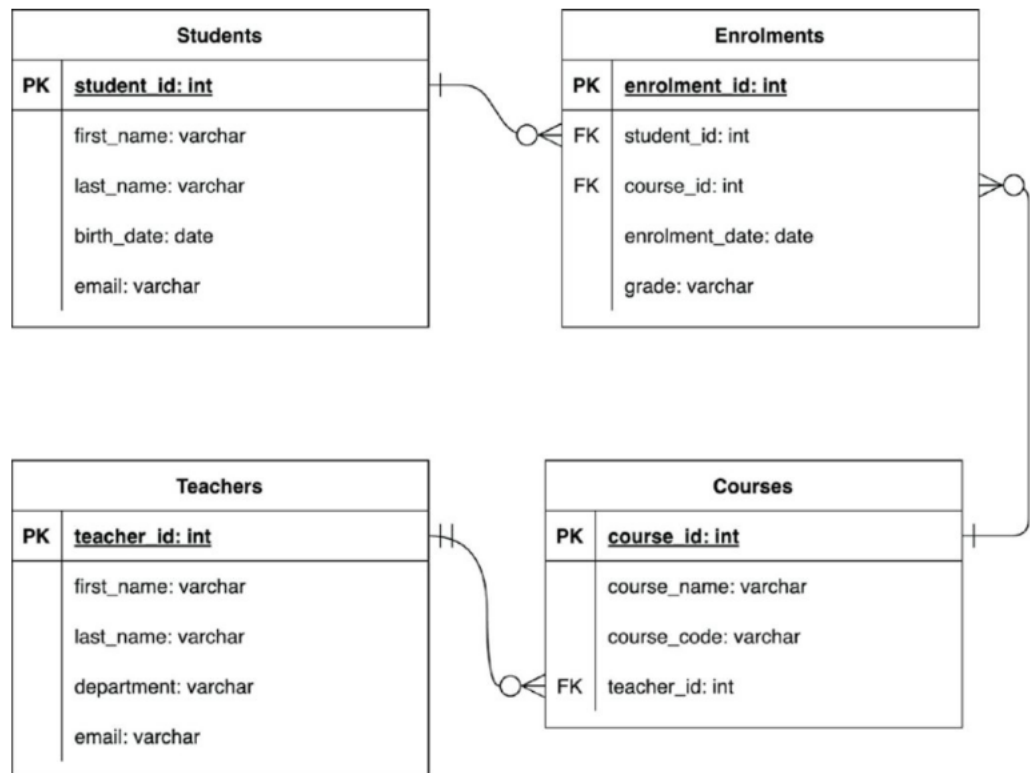
■ User interface with relevant logic side by side (credit: Sofia Cornu)



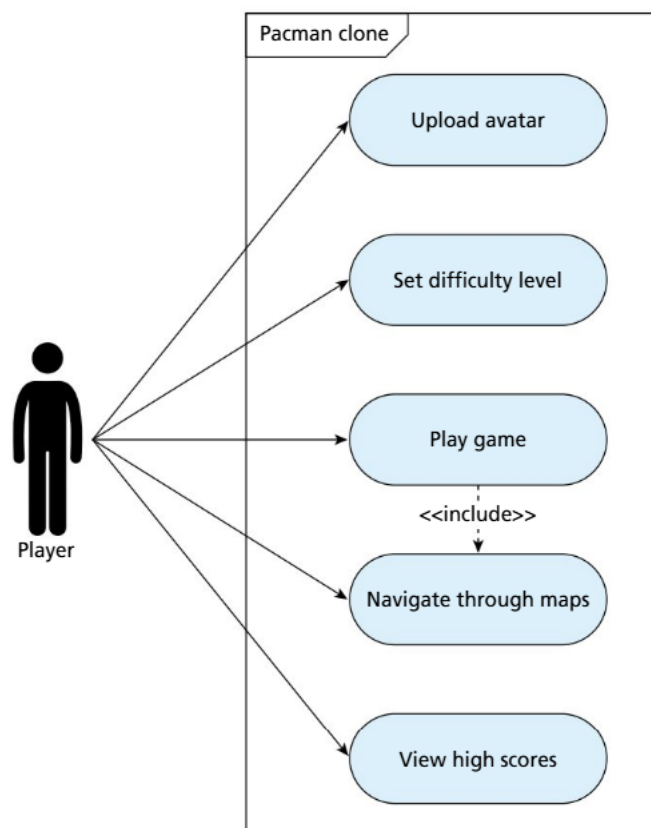
■ Data flow diagram (credit: former student)



■ Entity relationship diagram



■ UML class diagram with relationships



■ Use case UML

SETTINGS.JSON

```
{
  "language": "en",
  "timezone": "Asia/Bangkok",
  "dateFormat": "dd/mm/yyyy",
  "server": {
    "ipAddress": "192.168.1.100",
    "apiPort": 8080,
    "apiKey": "a1b2c3d4e5f6g7h8i9j0"
  }
}
```

■ Configuration file example

PSEUDOCODE FOR MAIN GAME LOOP IN PACMAN

```
METHOD run()
  Set game_running to True
  WHILE game_running is True
    Process events:
      If a quit event is detected, exit the game
    Clear the screen to the background color
    Draw the map with walls and food
    Move and draw Pacman
    Check and handle food consumption at Pacman's position
    For each ghost in ghosts array:
      Move and draw the ghost
      Check collision between the ghost and Pacman:
        If collision, print "Game over" with the score,
        set game_running to False
    Update the display to reflect any changes
    Control the frame rate with the clock
  END WHILE
END METHOD
```

■ Pseudocode example

Criterion D: Development

The development of the product must be consistent with the problem specification in criterion A, the planning in criterion B and the system overview developed in criterion C.

- The video should provide evidence of the functionality and give examples of the testing of the product.
- The development of the solution should justify the structure of the product and why it is appropriate, and demonstrate the techniques used to develop the product based on the algorithms constructed in criterion C. These techniques may include loops, data structures, existing libraries and the integration of software tools.
- The testing strategy should include testing for correctness, reliability and efficiency. The testing should be described and justified in the documentation, with supporting examples seen in the video.

The recommended word count for this criterion is 1000 words.

■ Criterion D assesses the development of the product (12 marks)

Marks	Description
0	The response does not reach a standard described by the descriptors below.
1–3	Constructs a product with very limited functionality. Constructs a product using no appropriate techniques to implement the algorithms. States the choices made to implement the algorithm. States the testing strategy used.
4–6	Constructs a product that has limited functionality. Constructs a product using at least one appropriate technique to implement the algorithms. Outlines the choices made to implement the algorithm. States the effectiveness of the testing strategy used.
7–9	Constructs a product that has partial functionality. Constructs a product that uses some appropriate techniques to implement the algorithms. Explains the choices made to implement the algorithm. Describes the effectiveness of the testing strategy used.
10–12	Constructs a fully functional product. Constructs a product that uses appropriate techniques to implement the algorithms. Evaluates the choices made to implement the algorithm. Justifies the effectiveness of the testing strategy used.

Clarification of development:

- **The implementation and coding of the algorithms:** Techniques in the criteria refer to the process of programming algorithms using code. The documentation should highlight key elements of code that are important for the efficient functioning of the algorithms. Any code presented in the solution should include relevant comments, be consistent and be readable. Code excerpts included in the documentation should be referenced to the full source code submitted as an appendix.
- **Functionality and testing:** The video should demonstrate the functionality of the product. The deployment of the testing strategy and its effectiveness should be described in the documentation with examples of the testing seen in the video.

IBDP Computer science guide

● Top tips!

Criterion D

- 1 The top band for this criterion shifts the focus from *explanation* and *description* to that of *evaluation* and *justification*. Why were the techniques you used appropriate? Why did you make the choices you did when implementing your algorithm(s)?
- 2 Why was your method of testing the solution optimal for the context? Show at least three of the outcomes of your test cases against this area of your application, covering both functional and structural testing.
- 3 Structure your criterion D with subheadings based on the success criteria, rather than as a narrative telling the history of the development of the solution. For each of your success criteria, present the case for the most appropriate techniques for completing it and your testing strategy used for it.
- 4 Ensure your video demonstrates testing of your product – not just for expected, but for unexpected, inputs. The video should also show that the software correctly processes the inputs, not just by showing the output produced, but by also showing that any internal state changes of the software have correctly occurred (for example demonstrate that a “save” function does produce a resulting change in the database).
- 5 When taking screenshots of your code to use in your documentation, include the line numbers. This makes it easier for you to refer to sections of your code in your writing.
- 6 The IB states that it is acceptable to improve code with GPT, but any use of GPT needs to be explicitly documented and the improvements need to be properly discussed and justified. Techniques must be recognized to receive marks for development and the choices made must be discussed.

● Common mistakes

Criterion D

- 1 **A very common issue is having limited discussion that explains why the approach was selected.** Justify why you programmed using the approach taken. There are several ways to do this, but the best is to contrast against alternative valid approaches and then explain the rationale of your decision.
- 2 **Remember your code that is submitted within the appendix forms part of criterion D.** Ensure your code is meaningfully stylized, structured and designed in accordance with the plans articulated in criteria B and C. One long file with thousands of lines in it, with functions and mainline code inter-mixed, no use of classes or meaningful modularization, will not likely score in the top band.
- 3 **The video is not for showing or explaining your code,** but for demonstrating the functionality of your application based on the success criteria. You only have five minutes; use them wisely.
- 4 **Videos longer than five minutes will not score more marks** as the moderator will likely stop watching at the five-minute mark. Anything beyond this will not be considered for your grade.

■ Checklist for criterion D

■ Video:

- 1 Your video addresses the problem described in criterion A.
- 2 A single test-run of the product is often sufficient to demonstrate the functionality, and then some examples of the testing strategy.
- 3 Ensure you showcase you have met every success criteria in your video (although full functionality can be achieved if your product is a working solution to an adequate problem specification, even if the success criteria have not been fully met).

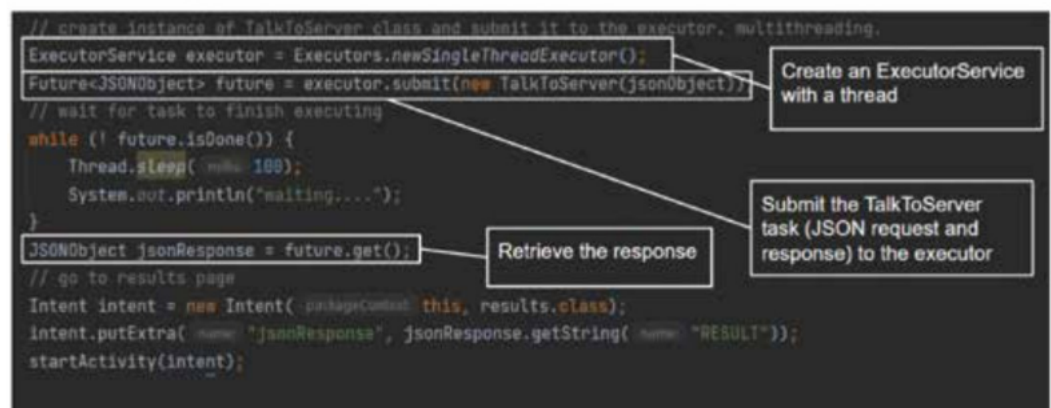
■ **Development document:**

- 4 Identify and address five to seven of your most advanced techniques / algorithms / data structures:
 - Discuss the techniques used to complete them.
 - Use screenshot excerpts of your code. Comments should be provided with code to explain their purpose.
 - Discuss alternative solutions you considered, where appropriate. Evaluate your choices.
- 5 Evidence of testing:
 - Functional testing
 - Structural testing
 - At least three separate test cases included (e.g. valid, extreme, invalid input data)
 - Evaluate the comprehensiveness of the testing
 - Justify your testing strategy.

For the evidence of testing, it is suggested you reproduce areas of the table from criterion C, with an additional column for 'results obtained'. If word count is a problem, you could simplify the table by numbering the tests in the criterion C table, and then referring to tests by their number when providing the results here in criterion D.

■ Examples for criterion D

Multithreading



■ Multithreading in "Mushroom: Friend or foe?" (credit: H Ng)

Explanation
In the Android application, multithreading is used to perform network operations, like fetching data from an API, without blocking the main UI thread. This approach keeps the UI responsive as network requests are handled in the background.
Justification
The advantage of this approach is that using separate threads to execute communication with the server ensures that the task can be completed concurrently in the main thread. As a result, the application stays responsive even if the request and response process takes a long time. An alternative approach could have been to use AsyncTask, however this is prone to memory leaks for beginner programmers. Kotlin Coroutines are a new feature designed to be a simple alternative to thread management, however these are not available for Java.
Testing
Structural tests (unit testing) and functional tests were conducted to ensure thread safety and correct data fetching. Tests under simulated slow network conditions confirmed the UI remains responsive.
Links to: success criterion 5

Calculating optimal route - Bitmasks Dynamic Programming and 2D array processing

This is the base case, all other elements in the array are set to infinity

Transition: Iterate over all possible masks, which represent all subsets of nodes

- Consider all other nodes "v" that haven't been visited yet

```
109 def find_min_cost_path(cost):
110     n = len(cost)
111     MAX_INT = 1 << n
112     dp = [[float('inf')] * n for _ in range(MAX_INT)]
113     dp[1][0] = 0
114     for mask in range(1, MAX_INT):
115         for u in range(n):
116             if mask & (1 << u):
117                 for v in range(n):
118                     if mask & (1 << v) and u != v:
119                         dp[mask][u] = min(dp[mask][u], dp[mask ^ (1 << u)][v] + cost[v][u])
120
121     min_cost = min(dp[MAX_INT - 1])
122     end_node = dp[MAX_INT - 1].index(min_cost)
```

mask XOR (1 << u): Removes node u from the set of visited nodes in mask

dp[mask XOR (1 << u)][v] + cost[v][u]: Total cost of reaching node u from the start, having first reached node v and then traveling from v to u

dp[mask XOR (1 << u)][v]: Gives the minimum cost of reaching node v from the start, having visited the set of nodes in mask minus node u

The minimum of these two values is then set as the new cost to reach u.

min_cost is the minimum value from the last row of the DP array, which contains the minimum costs of reaching the final node from all other nodes. end_node is the index of the minimum cost, which is the last node in the optimal path.

Set the initial bitmask to all 1s to represent that all nodes have been visited in the final state of the path.

Check if 'v' is part of the current mask (subset) and leads to the optimal path for 'end_node'. If yes, then it's part of the optimal route.

```
121     min_cost = min(dp[MAX_INT - 1])
122     end_node = dp[MAX_INT - 1].index(min_cost)
123     mask = MAX_INT - 1
124     path = [end_node]
125     while mask and end_node != 0:
126         for v in range(n):
127             if mask & (1 << v) and dp[mask][end_node] == dp[mask ^ (1 << v)][v] + cost[v][end_node]:
128                 path.append(v)
129                 mask ^= (1 << v)
130                 end_node = v
131                 break
132     return min_cost, list(reversed(path))
```

Update the mask by removing 'end_node' from the current subset, preparing for the next iteration of backtracking.

Set the current 'end_node' to 'v' as we move backwards through the path.

■ Calculating optimal route in “Vacation route planner” (credit: Edward Zhang)

Explanation
The 2D array <code>dp</code> is used to represent the states of the solution. Each state has a bitmask <code>mask</code> and a position <code>u</code> . The bitmask represents the subset of nodes visited so far, and the position represents the current node being processed. <code>dp[mask][u]</code> stores the minimum cost to reach node <code>u</code> after visiting the set of nodes represented by the bitmask <code>mask</code> (Datta, Subham). Refer to the screenshots above for a detailed explanation.
Justification
By using bitmask dynamic programming, the algorithm transforms the problem from a brute-force search of $O(N!)$ time complexity to $O(N^2 \times 2^N)$, which is feasible for N up to 15 to 20 (Kaçar, Kaan). Compared to more efficient heuristics such as Christofides’ algorithm or the nearest-neighbour heuristic, dynamic programming always guarantees the optimal solution, while heuristics only provide an approximation (GeeksForGeeks). An iterative approach was adopted because recursion could lead to stack overflow and increase computational complexity. The bottom-up dynamic programming approach builds up the solution from smaller subproblems to obtaining the final answer, which optimizes performance and memory usage.
Testing
Structural testing (with <code>unittest</code>) targeted the correctness of the bitmask operations and the 2D array update process. Tests showed that all possible states (mask combinations) correctly influence subsequent states. Functional tests simulated different node arrangements to ensure the algorithm consistently finds the optimal path.
Links to: success criterion 1

Criterion E: Evaluation

The evaluation of the product must be consistent with the problem specification and success criteria in criterion A.

The recommended word count for this criterion is 400 words.

■ Criterion E assesses the evaluation of the product

Marks	Description
0	The response does not reach a standard described by the descriptors below.
1–2	States the extent to which the success criteria were met. Describes improvements to the product.
3–4	Evaluates the extent to which the success criteria were met. Justifies improvements to the product.

IBDP Computer science guide

● Top tips!

Criterion E

- 1 The IB suggests that you may wish to include a table that lists all the success criteria together with an in-depth evaluation that addresses to what extent each criterion has been achieved by the solution.
- 2 Ensure you evaluate all of your success criteria individually and specifically.
- 3 When evaluating your success criteria and justifying the outcome, it is a good idea to refer back to the tests you carried out as evidence for this outcome.
- 4 Think of the improvements you would introduce if you were to develop a 2.0 version of your application. What features would you add?

Common mistakes

Criterion E

- 1 A common mistake with evaluation is overly simplistic and repetitive comments (e.g. "haven't run into bugs or errors"). The top band requires you to evaluate, not just state whether each of the success criteria is met. The *Guide* defines "evaluate" as "make an appraisal by weighing up the strengths and limitations".
- 2 Examples of inadequate improvements include:
 - a To include more colour / more data / more calculations / more functionality (unless very specifically justified).
 - b To add more functionality for success criteria that have not been met.
 - c To add a GUI interface for a solution that was developed as a CLI.
 - d To add functionality that is an essential (but missing) component of a fully functional solution (e.g. adding payment functionality to a web shop, or permanent storage for a database).

Checklist for criterion E

Evaluation:

- 1 Evaluate the success of **every** success criterion.
- 2 Come to a conclusion: Were the success criteria met, partially met or not met? Justify your reason for this conclusion, using the results of your tests to help evaluate this.

Improvements:

- 1 Improvements should be justified with reference to how they will address the specific issues.
- 2 Adequate improvement suggestions should be specific and actionable.

Examples for criterion E

Keep in mind that these examples are from the previous syllabus that required a client. The updated IA does not require a client. Rather than referring to your client feedback to evaluate each of your success criteria, you should refer back to your testing process as much as possible.

Evaluating success criteria in "Mushroom: Friend or foe?" (credit: H Ng)

Success criteria	Visuals	Evaluation
The program makes use of a graphical user interface to facilitate the input of mushroom features using drop-down boxes.	 The screenshot shows a web form titled "Enter Characteristics" with a light blue header. Below the title, there is a list of mushroom features, each followed by a drop-down menu. The features listed are: Bell, Cap surface (Fibrous), Cap colour (Brown), Bruises, Bruises, Odour (Almond), Gill attachment (Attached), Gill spacing (Close), Gill size (Broad), Gill colour (Black), Stalk shape (Enlarging), Stalk surface above ring (Fibrous), and Stalk surface below ring (Fibrous). Each feature has a corresponding drop-down arrow.	The client was able to enter mushroom features using drop-down boxes. The inputs that could be made through the drop-down boxes were all valid and were correctly saved in JSON string format. When asked if this criterion was satisfied, my client replied "absolutely" (Appendix 4).

■ Evaluating success criteria in “Automated grading tool” (credit: former student)

The program can automatically mark a multiple-choice paper with above 99% accuracy by comparing it to the mark scheme.



The program marked 30 out of 30 sample tests correctly. The client has confirmed that “Under good lighting conditions, this seems to work very well” (Appendix A.3).

■ Evaluating success criteria in “IBDP subject recommender” (credit: Sofia Cornu)

Application makes use of a graphical user interface to facilitate the inputting of personal information (the user can input this information through textboxes, checkboxes and choice boxes / drop-down select).



All of the data is either input by selecting from a drop-down select menu or by ticking checkboxes (as shown in visuals). My client stated that he thought the students found the inputting process “intuitive” and “quite self-explanatory” (Appendix E.1).

One of the improvements that my client and I discussed was the ability of the application to save features that had been input into the drop-down boxes, “even if the user were to go to a different page” (Appendix 4). As of now, the characteristics that have been entered disappear when the user clicks on the help page. To address this, an array can be made. When the help button is clicked, the chosen characteristics can be immediately saved to the array. Then, when the user returns to the “enter characteristics” page, they can seamlessly continue inputting additional characteristics that will be saved in the same array. The contents can then be sent to the neural network for processing.

Secondly, my client disliked that the application was “reliant on WiFi” to function. To fix this issue, the machine learning algorithm could be configured to run independently on the local system. The appropriate libraries (for example scikit-learn and NumPy) would have to be set up, and the h5 file containing the machine learning model would be downloaded on to my client’s phone.

Thirdly, my client suggested that entering 21 characteristics for each mushroom was “tedious”. To fix this, the importance of each feature could be calculated. A method to approach this is detailed by Fisher et al. Essentially, the value of a feature should be randomized and the accuracy of the model computed. This process can be repeated for each feature, and the ones with the lowest importance can be removed.

Finally, to enhance the educational aspect of the app, functionality can be added to provide guesses about the mushroom species. This can be achieved by creating a database that contains the characteristics of 50 common mushroom species. Each mushroom in the database would initially have a score of 0. When the user inputs characteristics, the app can compare them with the characteristics in the database. For each matching characteristic, the score of the corresponding mushrooms in the database can be incremented. The mushrooms with the highest scores can then be presented as guesses for the species of mushroom the user is trying to identify.

■ Suggested improvements in “Mushroom: Friend or foe?” (credit: H Ng)

Firstly, one of the improvements that my client and I discussed in our final interview was that of taking into account the possibility of “start[ing] the diploma with four higher levels”, which my client has said is “a conversation that [he has] often”. This could simply be done by adding another checkbox with this option, which would change the number of HLs SubjectSelect would pick.

Secondly, as my school has recently created a webpage with “details as to what are the assessments, what’s the difference between Higher and Standard Level” for each subject, my client suggested that having a link to this website from my SubjectSelect information page would be “something that could be really nice”.

Thirdly, another area for extension would be to break down some of the career field options I give (e.g. Physics, or anything related) into smaller more specific sub-sections, like “architecture, engineering and medicine”, and “work with people like Mr McArthur and Ms Edmunds” (my school’s counsellors) to implement the subject requirements for these careers in Switzerland, as these are, according to my client, the ones “that give (them) the most problems during the options process”.

Finally, another improvement to the project that I believe would add to its usefulness for students would be to include a screen that displays the weighting that each subject obtained (a simple change, as these weights are already all held in an arraylist). This would allow students to better understand why SubjectSelect chose the subjects it did.

■ Suggested improvements in “IBDP subject recommender” (credit: Sofia Cornu)

Frequently asked questions



■ Academic integrity

The IB provides the following guidance for your teachers when they are supporting you with this course:

The most important responsibility of all students is that the work they submit is their own. Students must understand and actively apply concepts related to academic integrity, such as authenticity, respect for intellectual property, and citing and referencing according to accepted systems. This includes instances where AI tools were used, such as to refine code. If students researched an existing solution then this must be clearly referenced and cited in a bibliography. Students must include the full source code of their solution in an appendix. Excerpts of code in the documentation should be referenced to the appended full source code.

IBDP Computer science teacher support material

The Guide also states that:

All work submitted to the IB for moderation or assessment must be authenticated by a teacher, and must not include any known instances of suspected or confirmed malpractice. Each student must confirm that the work is their authentic work and constitutes the final version of that work.

IBDP Computer science guide

Each school has its own academic integrity policy, which takes precedence over the generalized advice offered here. That said, advice follows below.

■ Using programming code from other sources

The use of programming code found through online sources is common in the Computer Science internal assessment. Sources of code may come from tutorials, example projects on Github, bug fixes found on Stack Overflow, generative AI, or any number of other sources.

When using code that has come from, or is adapted from, another source, then the equivalent to an 'in text' citation should be provided through the use of an in-code comment. The full bibliographical reference can then be included in the appendix to your written submission.

Consider the following Javascript example:

Java

```
// Adapted from "Arrow function expressions" (Mozilla)
// https://developer.mozilla.org/en-US/docs/Web/JavaScript/
// Reference/Functions/Arrow_functions
document.querySelector("[name='go']").
addEventListener("click", e=>{
    console.log("You clicked on ${e.target.name}");
});
```

■ Using generative AI

The IB has updated its academic integrity policy with respect to its views on the use of generative artificial intelligence tools. It is appendix 6 of the Academic Integrity Policy document available on the IB's website.

The overarching ethos comes from this statement:

The IB will not ban the use of AI software. The simplest reason is that it is the wrong way to deal with innovation. Over the next few years, the use of this kind of software will become as routine as calculators and translation programs. It is more sensible to adapt and teach students how to use these new tools ethically.

IB Academic integrity policy, 2023, page 53

From the perspective of how to apply the principle of ethical use of AI in your assessments, such as the internal assessment, the key statement is:

Students need to be aware that the IB does not regard any work produced – even only in part – by such tools to be their own. Therefore, as with any quotation or material from another source, it must be clear that any AI-generated text, image or graph included in a piece of work has been copied from such software. The software must be credited in the body of the text and appropriately referenced in the bibliography. If this is not done, the student would be misrepresenting content – as it was not originally written by them – which is a form of academic misconduct.

IB Academic integrity policy, 2023, page 54

As an IB student, you will need to keep in mind these points when using AI software:

- If you use the text (or any other product) produced by an AI tool – be that by copying or paraphrasing that text or modifying an image – you must clearly reference the AI tool in the body of your work and add it to the bibliography.
- The in-text citation should contain quotation marks, using the referencing style already in use by the school, and the citation should also contain the prompt given to the AI tool and the date the AI generated the text.

IB Academic integrity policy, 2023, page 54

■ Standard Level vs Higher Level

The IA is graded the same regardless of whether a student is at Standard Level or Higher Level, and both levels are graded against the same assessment criteria.

In fact, when IB moderators grade student IAs, they are not aware of whether a student is an SL or HL student.



■ Ethical guidelines

The *Guide* provides the following ethical guidelines for students completing the IA:

Given the nature of the internal assessment, students must take into account ethical problems and implications for undertaking research and developing the solution, for example ensuring the confidentiality and security of data. Wherever possible, original data should be used or collected by the student.

The following guidelines must be applied:

- Consent must be obtained from people who will be involved in the development of the computational solution before any investigation is begun.
- Written consent must be obtained from the owner of any existing system that is to be used as part of the internal assessment, for example when implementing a security analysis protocol on an existing system.
- All data collected must be stored securely in order to maintain confidentiality.
- Data collected can only be used for the computational solution. It must not be used for any other purpose without explicit permission.

IBDP Computer science guide